Digital Pendulum System

EXTERNAL INTERFACE TO THE REAL TIME KERNEL

33-005-3M5

Matlab 5 Version

Feedback
Feedback Instruments Ltd, Park Road, Crowborough, E. Sussex, TN6 2QR, UK.
Telephone: +44 (0) 1892 653322, Fax: +44 (0) 1892 663719.
email: feedback@fdbk.co.uk website: http://www.fbk.com

Manual: 33-005-3M5 Ed02 062000 *Printed in England by FI Ltd, Crowborough*
Feedback Part No. 1160–330053M5

Notes

THE HEALTH AND SAFETY AT WORK ACT 1974

We are required under the Health and Safety at Work Act 1974, to make available to users of this equipment certain information regarding its safe use.

The equipment, when used in normal or prescribed applications within the parameters set for its mechanical and electrical performance, should not cause any danger or hazard to health or safety if normal engineering practices are observed and they are used in accordance with the instructions supplied.

If, in specific cases, circumstances exist in which a potential hazard may be brought about by careless or improper use, these will be pointed out and the necessary precautions emphasised.

While we provide the fullest possible user information relating to the proper use of this equipment, if there is any doubt whatsoever about any aspect, the user should contact the Product Safety Officer at Feedback Instruments Limited, Crowborough.

This equipment should not be used by inexperienced users unless they are under supervision.

We are required by European Directives to indicate on our equipment panels certain areas and warnings that require attention by the user. These have been indicated in the specified way by yellow labels with black printing, the meaning of any labels that may be fixed to the instrument are shown below:

CAUTION - RISK OF DANGER

Refer to accompanying documents

CAUTION - RISK OF ELECTRIC SHOCK

CAUTION - ELECTROSTATIC SENSITIVE DEVICE

PRODUCT IMPROVEMENTS

We maintain a policy of continuous product improvement by incorporating the latest developments and components into our equipment, even up to the time of dispatch.

All major changes are incorporated into up-dated editions of our manuals and this manual was believed to be correct at the time of printing. However, some product changes which do not affect the instructional capability of the equipment, may not be included until it is necessary to incorporate other significant changes.

COMPONENT REPLACEMENT

Where components are of a ‘Safety Critical’ nature, i.e. all components involved with the supply or carrying of voltages at supply potential or higher, these must be replaced with components of equal international safety approval in order to maintain full equipment safety.

In order to maintain compliance with international directives, all replacement components should be identical to those originally supplied.

Any component may be ordered direct from Feedback or its agents by quoting the following information:

1. Equipment type
2. Component Value
3. Component reference
4. Equipment serial number

Components can often be replaced by alternatives available locally, however we cannot therefore guarantee continued performance either to published specification or compliance with international standards.

DECLARATION CONCERNING ELECTROMAGNETIC COMPATIBILITY

Should this equipment be used outside the classroom, laboratory study area or similar such place for which it is designed and sold then Feedback Instruments Ltd hereby states that conformity with the protection requirements of the European Community Electromagnetic Compatibility Directive (89/336/EEC) may be invalidated and could lead to prosecution.

This equipment, when operated in accordance with the supplied documentation, does not cause electromagnetic disturbance outside its immediate electromagnetic environment.

COPYRIGHT NOTICE

© Feedback Instruments Limited

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior permission of Feedback Instruments Limited.

ACKNOWLEDGEMENTS

Feedback Instruments Ltd acknowledge all trademarks.

IBM, IBM - PC are registered trademarks of International Business Machines.

MICROSOFT, WINDOWS 95, WINDOWS 3.1 , WINDOWS NT are registered trademarks of Microsoft Corporation.

MATLAB is a registered trademark of Mathworks Inc.

TABLE OF CONTENTS

1. Introduction 1-1

2. Structure of the Real-Time Kernel Dynamic Linked Library 2-1

3. External DLL Library 3-1

3.1. 16-bit DLL libraries 3-1
3.1. 1. Code and data segments type 3-4
3.1. 2. The module definition file 3-4
3.1. 3. Creating 16-bit DLL libraries 3-7
3.1. 3.1. Borland C++ v.3.1 3-7
3.1. 3.2. WATCOM C/C++ v.10.6 3-7
3.1. 3.3. Microsoft Visual C++ v. 1.5 3-9

3.2. 32-bit DLL libraries 3-9
3.2. 1. Code and data segments limitations 3-13
3.2. 2. Creating 32-bit DLL libraries 3-13
3.2. 2.1. Borland C++ v. 5.0 3-14
3.2. 2.2. WATCOM C/C++ v.10.6 3-14
3.2. 2.3. Microsoft Visual C++ v. 2.0 3-15

4. Exported Functions 4-1

4.1. ExternalController function 4-1

4.2. ExternalIPC function 4-5

5. Communication with MATLAB 5-1

5.1. Function LoadExtAlg 5-2

5.2. Function UnLoadExtAlg 5-4

5.3. Function GetExtAlgName 5-5

5.4. Function CallExtIPC 5-6

5.5. Function ExtAlgOneStep 5-7

6. Basic Steps to Implement an External Controller 6-1

6.1. Creating the ExternalController function 6-2

6.2. Creating the ExternalIPC function 6-2

6.3. TEsting in the MATLAB Environment 6-3

Contents

7. Example 1: Non-linear Stabilising Controller with Tuned Coefficients 7-1

8. Example 2: Off-line Testing of *Externalcontroller* 8-1

9. Example 3: 2-D Pid Controller 9-1

10. Example 4: Simulink Model for the 2-D Pid Controller 10-1

10.1. Crane mode Simulink model 10-1

10.2. Inverted pendulum mode Simulink model 10-13

1. Introduction

Although the Real-Time Kernel (RTK) for the inverted pendulum and the MATLAB interface create a complete self-contained environment for real-time experiments, its applicability is limited to a fixed number of embedded controllers.

The External Interface (EI) to the Real-Time Kernel was developed to extend the features of the control software. The interface provides the way in which user-designed controllers can be added to the RTK and implemented in real time. To more experienced users the External Interface offers an access path to the RTK and makes the control technology more open.

The external DLL library interface opens the way to add new controllers to the Real Time Kernel. The main advantage of the external interface is that the user does not need to know the implementation details of the real-time program.

The main part of this manual describes procedures for creating C-code DLL libraries. It should be noted that it is also possible to develop similar procedures for FORTRAN or other high level languages.

It is assumed that the reader is familiar with the following topics:

- RTK concept and the function library for inverted pendulum
- MATLAB and Simulink environment
- C programming language
- basic MS-Windows API (Application Programming Interface) functions
- Dynamic Linked Libraries concept.

It is also assumed that the reader is a licensed user of a C language compiler suitable to produce DLL executable files for MS-Windows/Windows95/Windows NT operating system and that the compiler is properly installed , and one knows how to use it.

This manual begins with a short description of the Real-Time Kernel and presents both its general structure and the principles of data exchange between modules of the system. The manual continues with a discussion of the structure of a DLL external library which is appropriate for the creation of external controllers and instructions how to compile and link DLL libraries are given.

In section 4 the shells of two basic functions exported from the external DLL for the inverted pendulum are described in detail - the *ExternalController* and *ExternalPC* The C-code template for the External DLL Library, which is a particularly useful tool for creating control algorithms, is also included. In section 5 a reference list of functions associated with the external DLL library is given and in section 6 a short overview of the external controller is given. Finally, three examples of controller implementation are shown.

There are three versions of RTKs available:

- for MATLAB version 4
- for MATLAB version 5 and Windows 95 operating system
- for MATLAB version 5 and Windows NT operating system.

The version for MATLAB version 4 works in MS-Windows and Windows 95. There are two versions of DLLs available: one for MATLAB 4 and another for MATLAB 5. The differences in external DLL implementation for both versions are caused by the differences between 16-bit and 32-bit DLL libraries. The C-sources of the DLL libraries contain statements specific for 16-bit or 32-bit libraries and code common for both versions. In this book it is assumed that the WIN32 pre-processor directive is declared when the user creates the 32-dit DLL library.

The C-source code able to distinguish 16-bit and 32-bit DLL has the following form:

```
#ifdef WIN32
    //  C-source specific for 32-bit DLL
    ..........
#else
    //  C-source specific for 16-bit DLL
    ..........
#endif
```

This manual describes the creation and application of DLL libraries for both 16-bit and 32-bit versions. Major differences in implementation and operation are distinguished in the separate chapters.

The example files of external DLL libraries are installed in the directory containing the software for Inverted Pendulum system in the EXTERNAL subdirectory.

The EXTERNAL directory contains the following files:

MK_BC16.BAT - file used to create 16-bit DLLs dedicated for Borland C/C++ compiler version 3.1

MK_BC32.BAT - file used to create 32-bit DLLs dedicated for Borland C/C++ compiler version 5.0

MK_MC16.BAT - file used to create 16-bit DLLs dedicated for Visual C/C++ compiler version 1.5

MK_MC32.BAT - file used to create 32-bit DLLs dedicated for Visual C/C++ compiler version 2.0

MK_WC16.BAT - file used to create 16-bit DLLs dedicated for Watcom C/C++ compiler version 10.6

MK_WC32.BAT - file used to create 32-bit DLLs dedicated for Watcom C/C++ compiler version 10.6

PD_2DPID.C - C-code used to create PID controller external DLL (see section 9 for details)

PD_2DPID.D16 - compiled 16-bit version of the PD_2DPID dedicated for MATLAB 4

PD_2DPID.D32 - compiled 32-bit version of the PD_2DPID dedicated for MATLAB 5

PD_2DPID.DEF - module definition file for the PD_2DPID.C,

PD_SS.C- C-code for creating DLL containing controller described in section 7

PD_SS.D16 - compiled 16-bit version of the PD_SS dedicated for MATLAB 4

PD_SS.D32 - compiled 32-bit version of the PD_SS dedicated for MATLAB 5

PD_SS.DEF - module definition file for the PD_SS.C

PD_TMF.C- template C-code

PD_TMF.D16 - compiled 16-bit version of the PD_TMF dedicated for MATLAB 4

PD_TMF.D32 - compiled 32-bit version of the PD_TMF dedicated for MATLAB 5

PD_TMF.DEF - module definition file for the PD_TMF.C

PD_2DSL.M - MATLAB GUI interface for the PD_2DPID DLL

PD_2DDN.M (for MATLAB 4) or **PD_2DDN.MDL** (for MATLAB 5) - Simulink model used to monitor the PD_2DPID.DLL external controller. It operates in *crane* mode

PD_2DSF.M - S-function used by the PD_2DDN Simulink model

PD_2DUP.M (for MATLAB 4) or **PD_2DUP.MDL** (for MATLAB 5) - Simulink model used to monitor PD_2DPID.DLL external controller. Operates in the inverted pendulum mode

PD_2DUSF.M - S-function used by the PD_2DUP Simulink model

The EXTERNAL directory contains a collection of compiled DLLs both for 16-bit and 32-bit versions. Remember to change the extensions of the selected ***.D16** and ***.D32** files in the EXTERNAL directory to the extension DLL before usage.

IMPORTANT !!!

The correct place to locate all DLLs is the MATLAB root directory. Execute the* matlabroot *command in the MATLAB Command Window to find the root directory of the MATLAB program. Remember that during experiments with external DLLs the MATLAB current directory must be the same as the MATLAB root directory. Use the* pwd *MATLAB command to check the name of the current directory. If necessary, use the* cd *command in the MATLAB Command Window to change the current directory.

2. Structure of the Real-Time Kernel Dynamic Linked Library

The general structure of the 16-bit control system is shown in Fig.2.1. The main part is the RTK DLL library (the *pd_dll.dll* file). It contains measurement procedures, digital filters, data acquisition buffer, built-in control algorithms, software to control actuators, internal excitation generator, set of controller parameters, MATLAB-to-RTK interface software and interface for external DLL libraries.

The RTK controls the flow of all signals from and to the process and contains functions performing analogue-to-digital and digital-to-analogue conversions. The RTK DLL library is excited by timer interrupts. The main part of the RTK is executed during interrupt time.

The Interface software performs data format conversion between the RTK library and the MATLAB environment (the interface procedures are included in the *pd_call.dll* file). The Inter-Process Communication Protocol is used to set parameters of the RTK and to read information from the RTK DLL library. Typically, the RTK library is monitored by a MATLAB program, but any other MS-Windows or Windows 95 program can be used in the supervisor layer. The RTK DLL library and the Interface software are designed to support External DLL libraries. The external library contains controller and interface software written by the user in the special form required by the RTK DLL library. As opposed to the *built-in* RTK algorithms, usage of the external DLL allows the possibility for the user to add new control algorithms, and provides the facility to extend the features of the control software.

The RTK designed for the MATLAB 5 and Windows 95 uses only one DLL: *pd_call.dll* file, which contains both the RTK and Interface Software (see Fig.2.2).

The RTK version for Windows NT operating system requires a special device driver (*RTKIO.SYS*) to allow access to the I/O address space. The access to the I/O space is limited in Windows NT and that is why the device driver is absolutely necessary. All “read” and “write” operations for the I/O Board are performed using this driver (see Fig.2.3).

The basic information required to implement a new algorithm is described in this section.

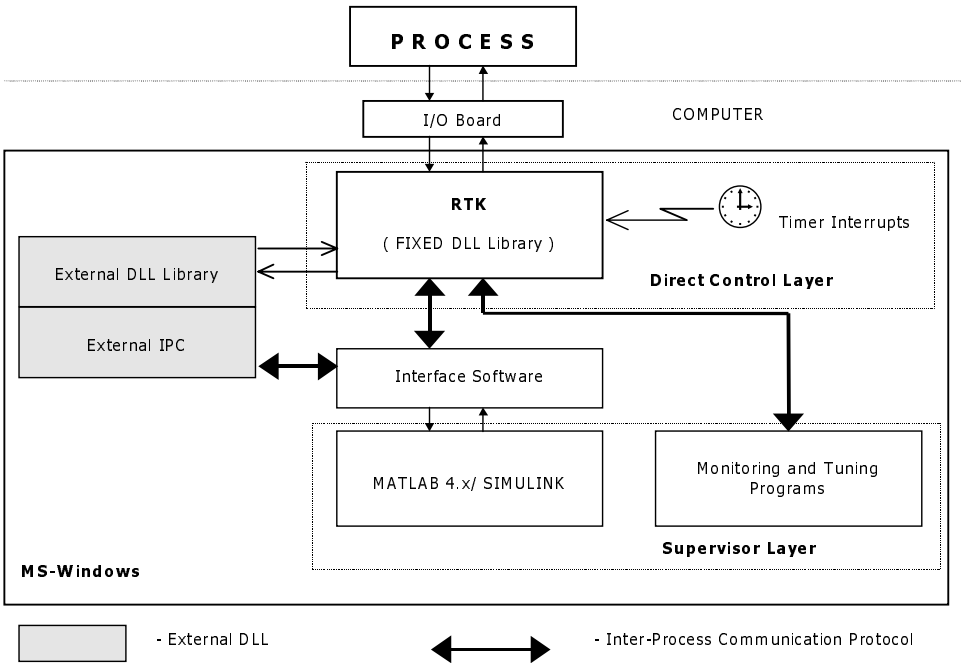


Fig.2.1. General structure of the control system for MATLAB 4.

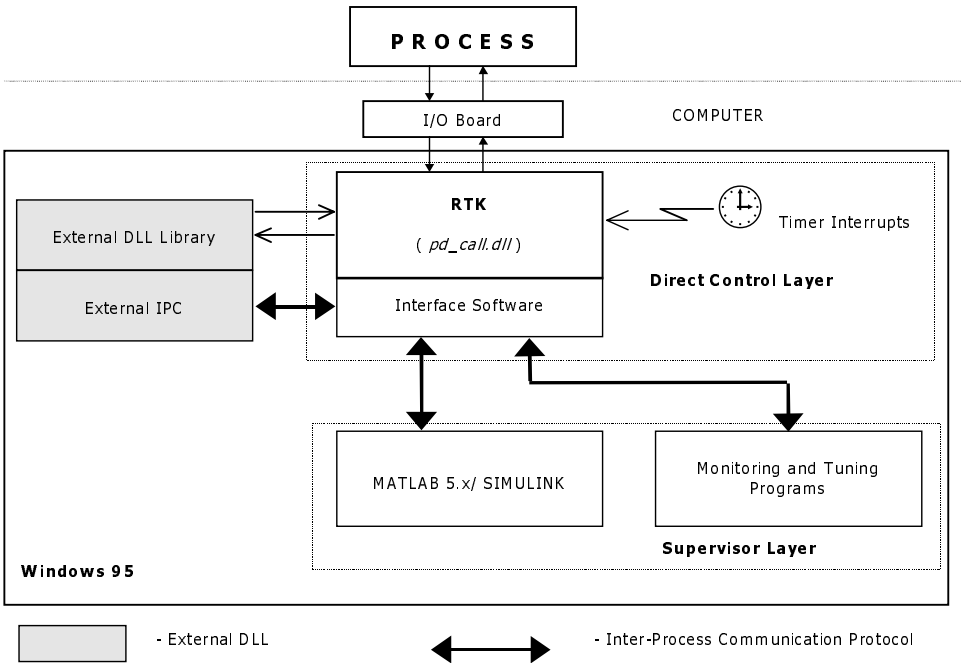


Fig.2.2. General structure of the control system for MATLAB 5 and Windows 95.

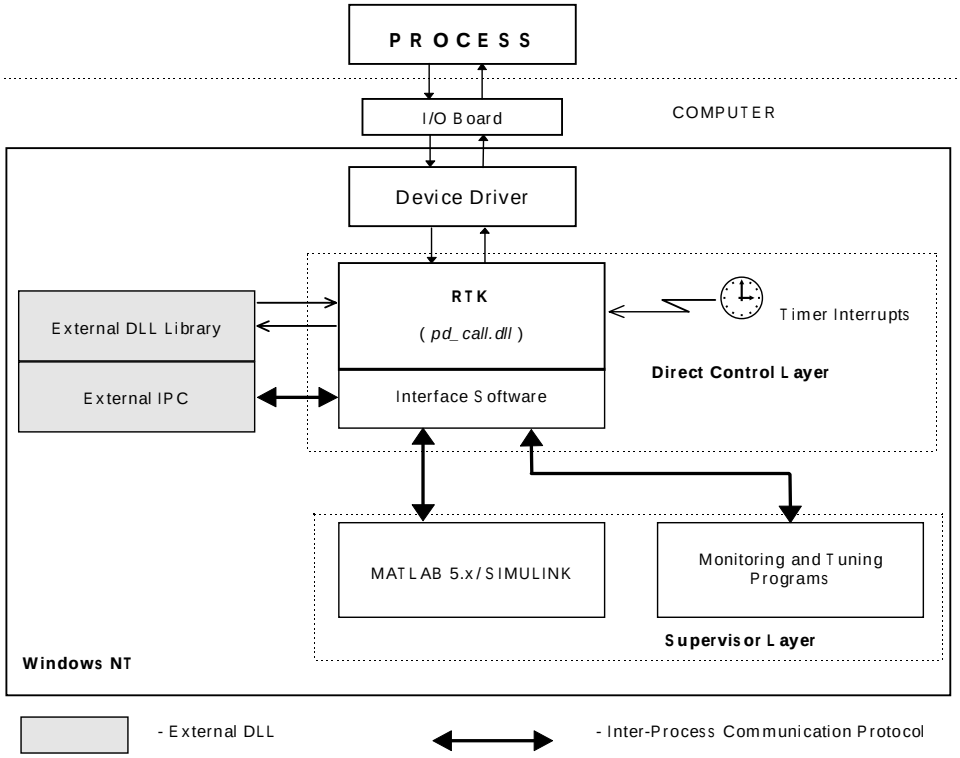


Fig.2.3. General structure of the control system for MATLAB 5 and Windows NT.

The RTK DLL library contains a set of built-in control algorithms selected by their numbers. The open loop control, rule-based controller, PID controller and state-space controller are implemented and included in the RTK. To start experiments, the RTK library must be loaded into the memory first, followed by the external DLL library.

The external DLL contains two main parts:

- the controller, and
- the interface software

To call the controller from the external DLL the **algorithm number 99** must be chosen (see Fig.2.4). In this case the RTK library calls the external library controller procedure during interrupt time. The RTK library passes the array of RTK parameters, current angle and velocity of the pendulum, position and velocity of the cart, current time value and reference value to the controller procedure as arguments. The controller procedure returns the control value for the DC drive. The interface software is used to exchange information between the external DLL library and the MATLAB environment (e.g. to set and get parameters or to choose the operating mode of the external library).

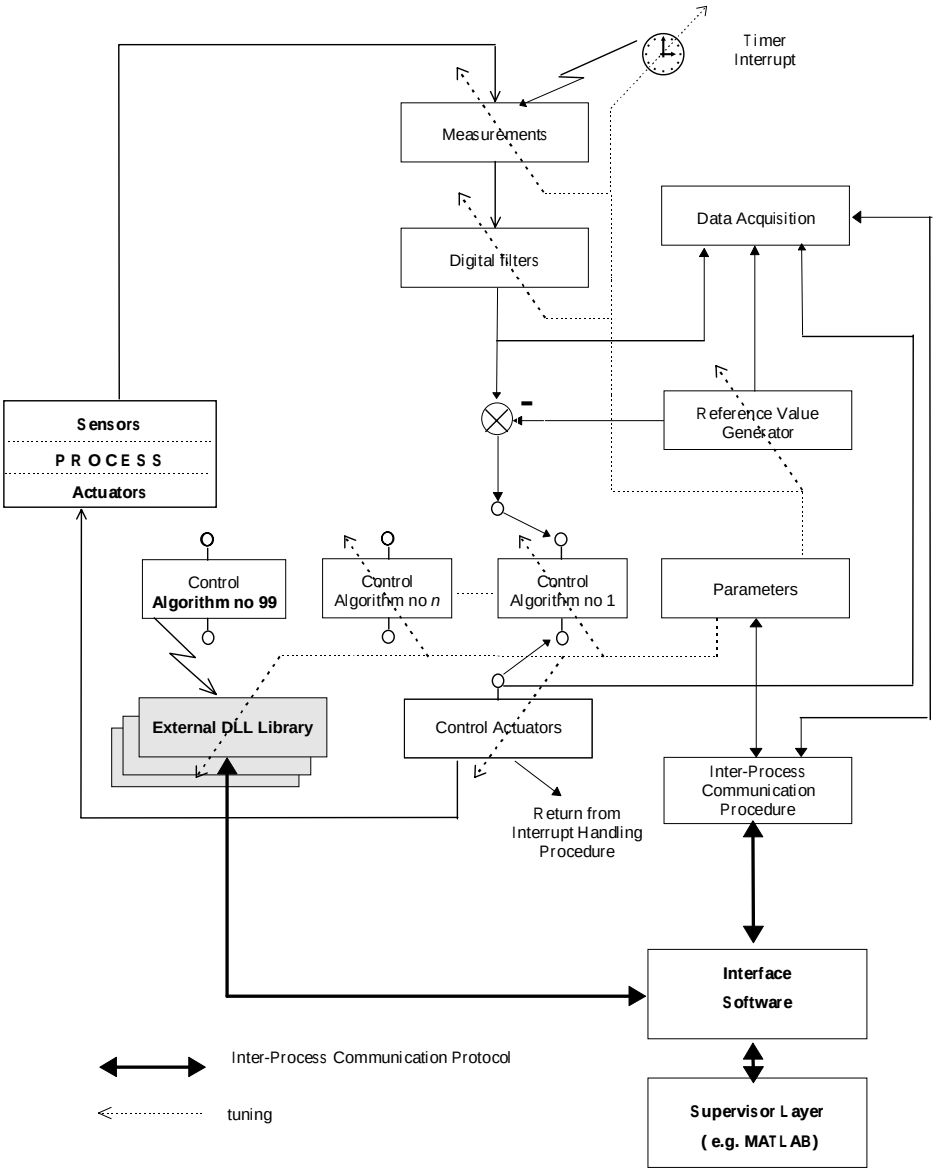


Fig.2.4. Block diagram of RTK.

The external DLL library can be removed from the memory when the experiment is finished. Another external DLL library can then be loaded end executed. However, it is not necessary to unload the RTK library from the memory when changing the executed external DLL. The external DLL operating mode is a fast way to test different versions of control algorithms.

3. External DLL Library

There are several factors that need to be understood in order to create your own external DLL libraries. This section describes the structure of such a library appropriate for the development of user-defined controllers.

There are two different structures of the 16-bit and 32-bit DLL libraries. They are described in the following sections.

3.1. 16-BIT DLL LIBRARIES

The main components needed to create the 16-bit DLL are shown in Fig.3.1.

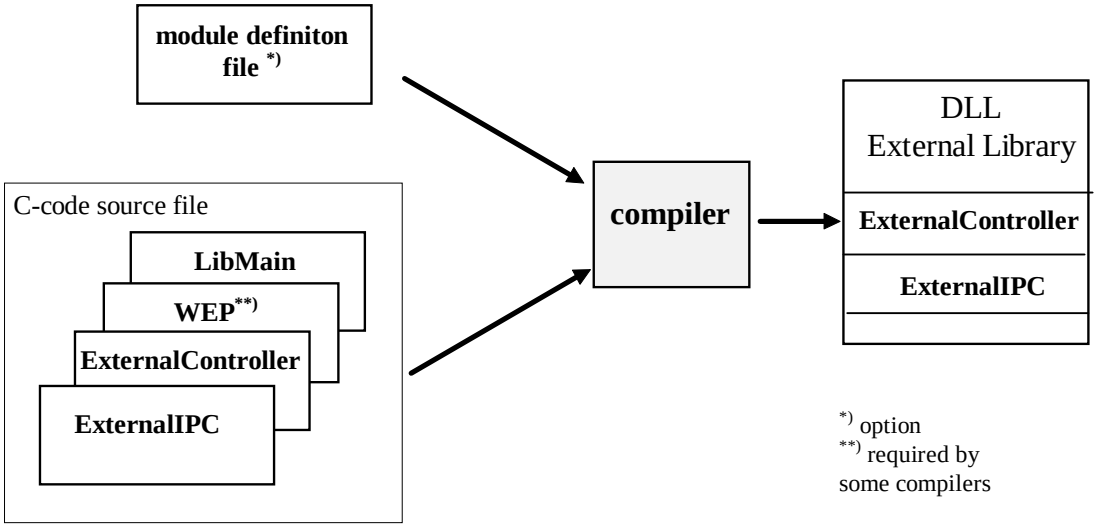


Fig.3.1. Procedure for creating the 16-bit external DLL library

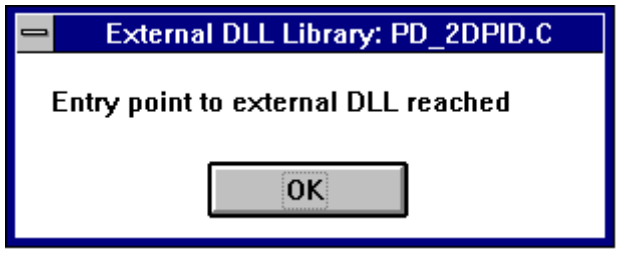
The C-source of an external DLL library must contain the following functions:

LibMain - function called by Windows during DLL initialisation. Use this function to initialise an external controller. For example, the following code creates a simple *LibMain* function:

```
#include <windows.h>
#include <stdio.h>
int FAR PASCAL LibMain( HINSTANCE hInstance,
                        WORD wDataSegment,
                        WORD wHeapSize,
```

```
                    LPSTR lpszCmdLine )
        {
          char msg_str[ 128 ];
          sprintf( msg_str, "External DLL Library: %s", __FILE__ );
          MessageBox( (HWND)NULL,
                  "Entry point to external DLL reached",
                  msg_str, MB_OK );

                /*                          */
                /* Insert your initialisation code here  */
                /*                          */

          return 1;   /* Indicate that the DLL was */
                  /* initialised successfully  */
}
```

Double *#include* directives include function prototypes. The definition of the string variable *msg_str*, the call of the *sprintf* function and the call of the *MessageBox* function are used to display the following window:



The string *__FILE__* contains the name of a current source file (*PD_2DPID.C* in the following example). The window appears every time the external DLL library is loaded into the memory (see description of the *LoadExtAlg* function below). If you do not want to display the window remove the first three statements in the function *LibMain*.

If your controller requires initialisation, insert the appropriate statements before returning from the *LibMain* function. For instance, if you want to allocate memory on the global heap, call the *GlobalAlloc* and the *GlobalLock* functions, etc. (compare Example 3 in section 8),

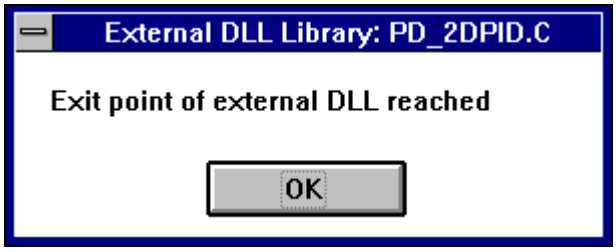
WEP - performs a cleanup of a dynamic-link library (DLL) before the library is unloaded. This function is called by MS-Windows. Although the *WEP* function was required for every dynamic-link library in the previous versions of the MS-Windows operating system,

for version 3.1, the *WEP* function is optional. Use this function to terminate the external controller operation safely. A simple example of a *WEP* function is shown below:

```
#pragma argsused
int FAR PASCAL WEP ( int bSystemExit )
{
  char msg_str[ 128 ];
  sprintf( msg_str, "External DLL Library: %s", __FILE__ );
  MessageBox( (HWND)NULL,
              "Exit point of external DLL reached",
              msg_str, MB_OK );

  /*                                        */
  /* Insert your termination code here      */
  /*                                        */

  return 1;
}
```

The `#pragma argsused` preprocessor directive disables the warning message *"Parameter 'parameter' is never used"*.

This *WEP* function displays the following window when the external DLL is removed from the memory (see description of the *UnloadExtAlg* function below):



The above message window is not obligatory. You can remove it if you do not want any additional messages during the removal of external DLL's from the memory. If your controller requires termination action, insert appropriate statements before returning from the *WEP* function. For instance, if you have allocated memory on the global heap call the *GlobalUnlock* and *GlobalFree* functions, etc.,

ExternalController - is called by the timer procedure at the beginning of each sample period and contains "the body" of the controller. The code of the *ExternalController* function is executed during interrupt time. See section 4 for implementation details,

ExternalIPC - function used to exchange information between an external DLL library and MATLAB/Simulink environment. This function is called by the interface software procedure *pd_call* from MATLAB, so it is not executed during the interrupt time. See section 4 for implementation details, of additional user-defined application-dependent functions.

3.1. 1. Code and data segments type

Because the procedure of an external controller is located in the external library and is executed during interrupt time, it must reside in the DLL, and its code memory segment must be specified as FIXED. Any data used by the external controller must be located in a FIXED data memory segment as well. The external controller can call ONLY the following API functions: *PostMessage, timeGetSystemTime, timeGetTime, timeSetEvent, timeKillEvent, midiOutShortMsg, midiOutLongMsg,* and *OutputDebugStr*.

The external inter-process procedure is not executed during interrupt time, so it may call any system functions.

The parameters of memory modules are usually specified in the module definition files.

3.1. 2. The module definition file

The module definition file usually contains information necessary to create a DLL library with appropriate parameters of the memory segments. An example of the module definition file is listed below.

```
LIBRARY        PD_2DPID
DESCRIPTION    'Pendulum External Algorithm for Real-Time Task'
EXETYPE         WINDOWS
CODE            PRELOAD FIXED
DATA            PRELOAD FIXED
HEAPSIZE        1024
STACKSIZE       1024
```

```
EXPORTS
                WEP                 @1 RESIDENTNAME
                ExternalController  @2 RESIDENTNAME
                ExternalIPC         @3 RESIDENTNAME
```

The above statements have the following meaning:

LIBRARY *PD_2DPID*

specifies the name of the library module. The linker uses the name to create *pd_2pid.dll* file,

DESCRIPTION *'Pendulum External Algorithm for Real-Time Task'*

inserts text into the application module. It is useful for inserting the version number or copyright information,

EXETYPE *WINDOWS*

specifies the header type of a generated executable file. It must be defined as WINDOWS in the case of executable files intended for the MS-Windows environment,

CODE *PRELOAD FIXED*

DATA *PRELOAD FIXED*

indicate the attributes of the code and data segments. The *PRELOAD* option specifies that the segment is loaded when the module is loaded for the first time. The *FIXED* option indicates that the segment remains at a fixed memory location and can be used by interrupt procedures,

HEAPSIZE *1024*

specifies the number of bytes needed by an application for its local heap. The application uses the local heap whenever it allocates local memory. The size of the local heap must be large enough to allocate memory regions required by the external controller. The default local heap size is zero. The minimum size is 256 bytes. The heap size must not exceed 65536 bytes. If you do not intend to allocate memory or you do not want to use local heap this statement may be omitted. In this case you are still able to use the global heap,

STACKSIZE *1024*

specifies the number of bytes needed by the DLL for its local stack. A DLL uses the local stack whenever it makes function calls,

```
EXPORTS
                WEP                 @1 RESIDENTNAME
                ExternalController  @2 RESIDENTNAME
                ExternalIPC         @3 RESIDENTNAME
```

specifies the names and attributes of the functions to be exported to other applications. The EXPORTS keyword marks the beginning of definitions. It can be followed by any number of export definitions, each in a separate line. Three functions: *WEP, ExternalController* and *ExternalIPC* must be implemented in C-code and exported. Additional functions can be exported if necessary. The attribute *@number* defines the location of the function name in its application string table. The attribute *RESIDENTNAME* indicates that the function must be resident all the times. If necessary, you may add your own functions to the *EXPORTS* directive.

The module definition file can be created by any text editor. The typical extension of this file is DEF. The example given above is stored in the file *pd_2dpid.def* (the \EXT_DLL directory on the installation diskette).

The use of a module definition file depends on the linker type. Some linkers do not require a module definition file to create a DLL and to correctly set parameters of memory segments. Consult the description of your compiler and linker for details.

3.1. 3. Creating 16-bit DLL libraries

In this section we explain how to compile and link DLL libraries using Borland C++ v.3.1, WATCOM C/C++ v.10.6 or Microsoft Visual C++ ver.1.5 compilers. If you want to use another compiler consult the documentation for proper use of the compiler and linker.

The procedure is illustrated in figure 3.1.

3.1. 3.1. Borland C++ v.3.1

To obtain a DLL executable file execute the following command:

```
bcc -ml -3 -f287 -WD -P- -IE:\BCPP\INCLUDE -LE:\BCPP\LIB
filename.c
```

Replace the *E:\BCPP* strings by the path to root directory of the Borland C++ compiler. Replace the *filename.c* by the name of your C-code source file. The *bcc* program performs both compiling and linking.

The presented command creates MS-Windows executable DLL file (option *-WD*), with code generated for 80386 microprocessor and for the 80287/80387 floating point unit (options *-3* and *-f287*). The command doesn't force compilation in C++ mode (option -P-). It defines the names of directories containing header files and libraries (options -I and -L). The code is generated in the large memory model (option *-ml*).

The *bcc* program requires a module definition file accessible during the linking process. The module definition file must have the same name as the source file. It must have the extension DEF. For instance, if the name of your C-code source file is *pd_2dpid.c* the module definition file must be stored in the file *pd_2dpid.def*.

3.1. 3.2. WATCOM C/C++ v.10.6

The WATCOM C/C++ compiler does not require a module definition file. Instead, an additional file with options for the linker must be created. The following batch file compiles and links DLL library.

```
rem COMPILE
wcc filename.c -ml -zu -bd -bt=windows

rem PREPARE FILE FOR LINKER
```

```
@echo NAME filename                     > wc_10.lnk
@echo FILE filename.obj                >> wc_10.lnk
@echo option stack=8k                  >> wc_10.lnk
@echo option heapsize=8k               >> wc_10.lnk
@echo system windows_dll               >> wc_10.lnk
@echo segment TYPE CODE PRELOAD FIXED >> wc_10.lnk
@echo segment TYPE DATA PRELOAD FIXED >> wc_10.lnk
@echo export WEP                       >> wc_10.lnk
@echo export ExternalController        >> wc_10.lnk
@echo export ExternalIPC               >> wc_10.lnk

rem LINK
wlink @wc_10.lnk

del wc_10.lnk
```

The compiler *wcc* compiles the *filename.c* file in the large memory model (*-ml*), sets the SS segment register to a different value than the DGROUP (*-zu*) and prepares the object file for DLL in the MS-Windows system (*-bd* and *-bt=windows*). The *wc_10.lnk* file contains options for the linker. The *NAME filename* and *FILE filename.obj* set the name of DLL library and the name of the object file. The *option stack=8k* and *option heapsize=8k* sets the size of the stack and the size of the local heap. The *system windows_dll* directive is used to produce the DLL for MS-Windows library. Two *segment* directives set options of the data and code segments. The *export* directives export function names.

3.1. 3.3. Microsoft Visual C++ v. 1.5

To obtain a DLL executable file execute the following command:

cl /G2 /FPi87 /W3 /ALw /Gx- /O2 /D "NDEBUG" /Gw /c file.c

link /NOD /NOE /PACKC:61440 /ALIGN:16 file.obj, file.dll, , oldnames libw mdllcew, file.def

The *cl* command performs compilation. The *link* command links the DLL library. Replace the *file* string with the name of your C-file in both commands.

3.2. 32-BIT DLL LIBRARIES

The components needed to create the 32-bit DLL are shown in Fig.3.2.

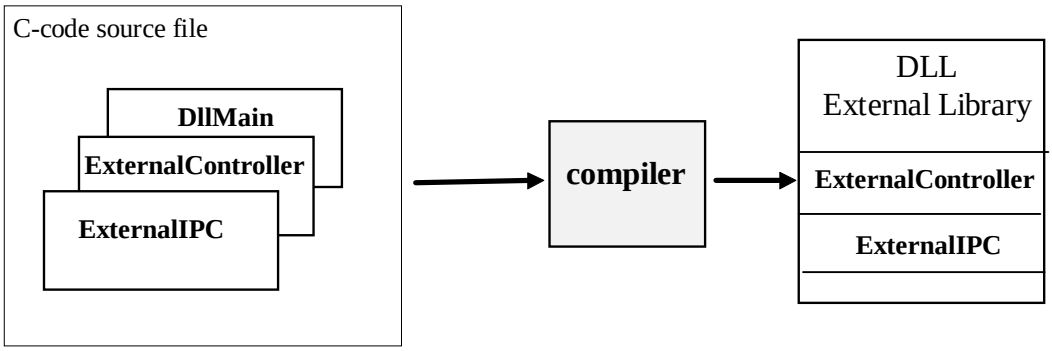


Fig.3.2. Procedure for creating the 32-bit external DLL library

The C-source code of the 32-bit external DLL library must contain the following functions:

DllMain - function called by Windows95 or Windows NT when the DLL library is:

- loaded for the first time by a given process, or when
- the DLL is unloaded by a given process,
- a thread is created in a process that has already loaded this DLL,
- a thread is exiting cleanly in a process that has already loaded this DLL.

The two last cases are not used by the RTK.

Use this function to initialise and perform a cleanup for an external controller. For example, the following code creates a simple DllMain function:

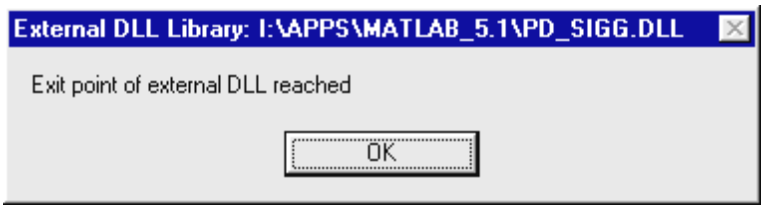
```
#include <windows.h>
#include <stdio.h>


BOOL WINAPI DllMain( HINSTANCE hDLLInst, DWORD fdwReason,
                     LPVOID lpvReserved )
{
  char msg_str[ 150 ];
  static char ModuleName[ 128 ];

  switch (fdwReason)
  {
    case DLL_PROCESS_ATTACH:
    // The DLL is being loaded for the first time by a given
    // process. Perform per-process initialization here. If
    // the initialization is successful, return TRUE; if
    // unsuccessful, return FALSE.

    GetModuleFileName( hDLLInst, ModuleName,
                       sizeof( ModuleName ) - 30 );
    sprintf( msg_str, "External DLL Library: %s",
             ModuleName );
    MessageBox( (HWND)NULL,
          "Entry point to external DLL reached",
           msg_str, MB_OK );

/*                                        */
/* Insert your initialization code here  */
/*                                        */


    break;
```

```
            case DLL_PROCESS_DETACH:
            // The DLL is being unloaded by a given process. Do any
            // per-process clean up here, such as undoing what was
            // done in DLL_PROCESS_ATTACH. The return value is
            // ignored.

            sprintf( msg_str, "External DLL Library: %s",
                     ModuleName );
            MessageBox( (HWND)NULL,
                        "Exit point of external DLL reached",
                        msg_str, MB_OK );

        /*                                          */
        /* Insert your termination code here        */
        /*                                          */

            break;

            case DLL_THREAD_ATTACH:
            // A thread is being created in a process that has
            // already loaded this DLL.  Perform any per-thread
            // initialization here. The return value is ignored.
            //
            // NOT USED in external DLLs

            break;

            case DLL_THREAD_DETACH:
            // A thread is exiting cleanly in a process that has
            // already loaded this DLL.  Perform any per-thread
clean
            // up here. The  return value is ignored.
            //
            // NOT USED in external DLLs

            break;
```

```
    }
    return TRUE;
  }
```

Double *#include* directives include function prototypes. The definition of string variable *msg_str*, the call of the *sprintf* function and the call of the *MessageBox* function are used to display the following window during loading the DLL:

and the following window during removing the DLL from the memory



The window appears every time the external DLL library is loaded to the memory (see description of the *LoadExtAlg* function below). If you do not want to display the window remove statements in the function *DllMain*.

If your controller requires initialisation, insert appropriate statements before the return from the *DllMain* function in the `DLL_PROCESS_ATTACH` case statement. For instance, if you want to allocate memory on the global heap call the *GlobalAlloc* and the *GlobalLock* functions, etc. (compare Example 3 in section 8),

The statements below the `DLL_PROCESS_DETACH` value perform a cleanup of a dynamic-link library (DLL) before the library is unloaded. Use this function to terminate the external controller operation safely. For instance, if you have allocated memory on the global heap call the *GlobalUnlock* and *GlobalFree* functions, etc.,

The *DllMain* function returns a BOOL value which is used only when *fdwReason == DLL_PROCESS_ATTACH*. The *TRUE* return value is used to signify that the DLL should remain loaded. The *FALSE* value is used to signify that the DLL should be immediately unloaded. For all other values of *fdwReason*, the return value is ignored.

ExternalController - is called by the timer procedure at the beginning of each sample period and contains "the body" of the controller. The code of the *ExternalController* function is executed during interrupt time. See section 4 for implementation details,

ExternalIPC - function used to exchange information between an external DLL library and MATLAB/Simulink environment. This function is called by the interface software procedure *pd_call* from MATLAB, so it is not executed during the interrupt time. The external inter-process procedure is not executed during interrupt time, so it may call any system functions. See section 4 for implementation details, of additional user-defined application-dependent functions.

3.2. 1. Code and data segments limitations

The procedure of an external controller is located in the external library and is executed during interrupt time. The external controller can call ONLY the following API functions: *PostMessage, timeGetSystemTime, timeGetTime, timeSetEvent, timeKillEvent, midiOutShortMsg, midiOutLongMsg,* and *OutputDebugStr*.

3.2. 2. Creating 32-bit DLL libraries

In this section we explain how to compile and link DLL libraries using Borland C++ v.5.0, WATCOM C/C++ v.10.6 and Microsoft Visual C++ ver.2.0 compilers. If you want to use another compiler consult the documentation for proper use of the compiler and linker.

The procedure is illustrated in figure 3.2.

3.2. 2.1. Borland C++ v. 5.0

To obtain a 32-bit DLL executable file execute the following command **(see the *mk_bc32.bat* file** on the installation disk):

```
set BCR=J:\BC5

%BCR%\bin\Bcc32 -P- -c -I%BCR%\INCLUDE -DDllMain=DllEntryPoint
-DWIN32 filename.c

%BCR%\bin\Tlink32 -L%BCR%\LIB -Tpd -aa -c -x %BCR%\LIB\c0d32.obj
filename.obj, filename, filename,
%BCR%\LIB\import32.lib %BCR%\LIB\cw32.lib
```

Replace the definition if the *BCR* variable by the path to root directory of the Borland C++ compiler. Replace the *filename* by the name of your C-code source file (without extension).

The presented commands creates Windows 32-bit executable DLL file.

3.2. 2.2. WATCOM C/C++ v.10.6

The following batch file compiles and links 32-bit DLL **library (see the *mk_wc32.bat* file** on the installation disk):

```
wcc386 filename.c -ih:\lang\watcom\h;h:\lang\watcom\h\nt -w4 -e25
-zq -otexan -bd -fp3 -4r -bt=nt -mf -dWIN32

wlink SYS nt_dll op m op maxe=25 op q op symf N filename
F filename.obj

wlib -n -b filename.lib +filename.dll
```

The compiler *wcc386* compiles the *filename.c* file. Replace the `h:\lang\watcom` string by the name of your WATCOM C/C++ root directory. Replace the *filename* string by the name of your C-file.

3.2. 2.3. Microsoft Visual C++ v. 2.0

To obtain a 32-bit DLL executable file execute the following **command (see the *mk_mc32.bat* file** on the installation disk):

```
set MC_ROOT=g:\apps\msvc20

%MC_ROOT%\bin\cl /nologo /MT /W3 /GX /YX /O2 /D "WIN32"
/D "NDEBUG" /D "_WINDOWS" /I%MC_ROOT%\include
/c filename.c

@echo %MC_ROOT%\lib\kernel32.lib          > mc_opts.lnk
@echo %MC_ROOT%\lib\user32.lib           >> mc_opts.lnk
@echo %MC_ROOT%\lib\libcmt.lib           >> mc_opts.lnk
@echo %MC_ROOT%\lib\oldnames.lib         >> mc_opts.lnk

@echo /NOLOGO /SUBSYSTEM:windows /DLL    >> mc_opts.lnk
@echo /INCREMENTAL:no /MACHINE:I386      >> mc_opts.lnk

%MC_ROOT%\bin\link @mc_opts.lnk filename.obj
```

Replace the definition if the *MC_ROOT* variable by the path to root directory of the Visual C++ compiler. Replace the *filename* by the name of your C-code source file (without extension).

Notes

4. Exported Functions

Two functions from the external DLL must be implemented and exported. The first one is named ***ExternalController***. It contains "the body" of the implemented controller. The interrupt handling procedure from RTK calls *ExternalController* at each sampling period. The *ExternalController* procedure is accessed at interrupt time, so it may not call any other operating system procedure except for: *PostMessage, timeGetSystemTime, timeGetTime, timeSetEvent, timeKillEvent, midiOutShortMsg, midiOutLongMsg,* and *OutputDebugStr*.

The second function is named ***ExternalIPC***. It is responsible for inter-process communication between external DLL and MATLAB. You may use any system call inside the body of *ExternalIPC* function as it is not executed during the interrupt time.

NOTICE:

***the C-file containing template C-code for an external DLL library is stored on installation disk as* pd_tmp.c .**

4.1. EXTERNALCONTROLLER FUNCTION

The *ExternalController* function is executed during each sampling period. You can use input arguments passed to this function to calculate control for the D/A converter. The function is called by RTK when:

the external DLL library was loaded successfully (use *pd_call('LoadExtAlg', DLL_name)* in MATLAB command window) and,

the algorithm number was set to 99 (execute *pd_call('SetAlgNo', 99)* in the MATLAB command window).

Remember that the ExternalController function is executed at the interrupt time. Do not call system procedures except for: PostMessage, timeGetSystemTime, timeGetTime, timeSetEvent, timeKillEvent, midiOutShortMsg, midiOutLongMsg, and OutputDebugStr.

The shell of the *ExternalController* is listed below. Notice that the macro definition WIN32 is used to distinguish statements dedicated for 16-bit and 32-bit DLLs (or for MATLAB 4 and MATLAB 5).

```
#ifdef WIN32  /* for MATLAB 5 */
DLLEXPORT void __stdcall ExternalController(
        double *Param,
        double PendPos,
        double PendVelocity,
        double CartPos,
        double CartVelocity,
        double Time,
        double DesValue,
        double *Control )
#else         /* for MATLAB 4 */
void FAR PASCAL ExternalController(
        double far *Param,        /* Parameters of the controller
*/
        double far  PendPos,      /* Angle [rad]
*/
        double far  PendVelocity, /* Angular velocity [rad/s]
*/
        double far  CartPos,      /* Cart's position [m]
*/
        double far  CartVelocity, /* Cart's velocity [m/s]
*/
        double far  Time,         /* Time [s]
*/
        double far  DesValue,     /* Desired value of the cart [m]
*/
        double far *Control )     /* Calculated control value
*/
#endif
{
  double ctrl = 0.0;
  /*                                                    */
  /* Put the source code of your controller here */
```

```
  /* Remember that the control value for          */
  /* the DC motor must be returned as:            */
  /*   *Control = value;                          */

  /* Check limits of the control value */
  if(ctrl <= -1.0 ) ctrl = -1.0;
  if(ctrl >= +1.0 ) ctrl = +1.0;

  *Control = ctrl;
}
```

The *ExternalController* function has eight input arguments:

Param - pointer to the array containing twenty parameters. The parameters may be set by the *pd_call('SetP', new_array)* function from the MATLAB command window. If the values of the parameters are required in the MATLAB workspace execute the *pd_call('GetP')* command. Use both functions to tune parameters during real-time experiments, in the *on-line* mode,

PendPos - angular position of the pendulum. The angle is expressed in radians. The down position is equal to $\pm\pi$ rad. The upright position is equal to 0 rad. Notice, that before starting your experiments you must execute *pd_call('ResetEncoder')* function when the cart is placed in the centre of the rail and the pendulum is in its down position (the cart and the pole are motionless). The angle changes by $\pm 2\pi$ each full revolution of the pendulum. For instance, if the pendulum has performed one full revolution the angle of down position is equal to $+3\pi$ rad or to -3π rad and the angle of the upright position is equal to $+2\pi$ rad or to -2π rad. The sign of the angular position depends on the direction of motion,

PendVelocity - angular velocity of the pendulum expressed in rad/sec. The velocity is calculated according to the following formula:

$$angular_velocity = \frac{pend_position_{current} - pend_position_{previous}}{sampling_period}.$$

CartPos - position of the cart on the rail relative to the position set by the execution of *pd_call('ResetEncoder')* command. The position is expressed in meters. The working range of the rail is approximately equal to ±0.6 m,

CartVelocity - linear velocity of the cart expressed in m/sec. The velocity is calculated according to the following formula:

$$linear_velocity = \frac{cart_position_{current} - cart_position_{previous}}{sampling_period}.$$

Time - time calculated from the beginning of the experiment or from the executing of the *pd_call('ResetTime')* command. Time is expressed in [s],

DesValue - desired position of the cart on the rail. This value can be set by the *pd_call('SetDesPosition', new_des_position)* function and get by the call of the *pd_call('GetDesPosition)* function,

Control - pointer to control value for the DC drive. The control must be calculated by the *ExternalController* function and assigned to variable pointed to by *Control* pointer in the expression:

```
*Control = new_control_value;
```

The range of control value is ±1.0. The control equal to +1.0 yields full output action in one direction, and the control equal to -1.0 yields full output action in the opposite direction. The calculated control value is stored in the data buffer located in RTK and may be accessed by the *pd_call('GetHistory')* function.

In the example given above the *ExternalController* function does not correspond to any particular controller. It performs only range checking and assigning of control value. The full implementation of a simple controller is shown in Example 1.

The *ExternalController* function can be called directly from MATLAB environment. In this case the body of the function is not executed at the interrupt time (nonreal-time mode). Such call can be useful as *off-line* testing before starting real-time experiments. See description of the *GetExtAlgOneStep* function in section 5.

4.2. EXTERNALIPC FUNCTION

The *ExternalIPC* function can be used to exchange information between the MATLAB workspace and the external DLL library. Typically, it sets the parameters of the external controller or gets the value of any variable located inside the external DLL.

Figure 4.1 illustrates the principle of operation of the external IPC.

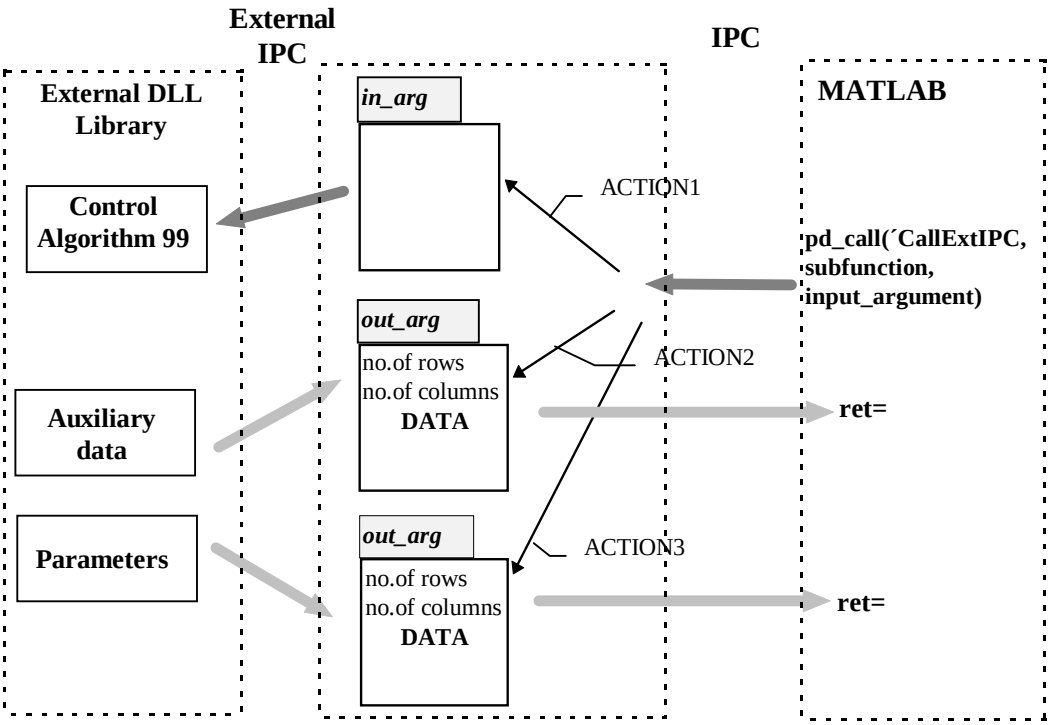


Fig.4.1. ExternalIPC operation principle

An example of the *ExternalIPC* function is shown below:

```
#ifdef WIN32
DLLEXPORT void __stdcall ExternalIPC(
            char    *str,
            double *in_arg,
            double *out_arg )
#else
void FAR PASCAL ExternalIPC(
            char    far *str,        /* Function          */
```

```
            double far *in_arg,     /* Input arguments    */
            double far *out_arg )   /* Output values      */
#endif
{
  int i;
  if( strcmp( str, "ACTION_1" ) == 0 )
  {
/* Put here code for "ACTION_1"  */
/* Example: return single value  */

    *out_arg = *(out_arg + 1) = 1.0;
    *(out_arg + 2) = 1.0;
    return;
  }

  if( strcmp( str, "ACTION_2" ) == 0 )
  {
/* Put here code for "ACTION_2"  */
/* Example: return 10 elements   */

    *out_arg = 1.0;  *(out_arg + 1) = 10.0;
    for( i = 0; i < 10; i++ )
      *(out_arg + 2 + i) = sin( i );
    return;
  }
*out_arg = *(out_arg + 1) = 1.0;
*(out_arg + 2) = -1.0;
}
```

The *ExternalIPC* function has three input arguments:

str - string variable. Typically, the *str* variable is used to distinguish different actions performed by the *ExternalIPC* function. Actions are initialised by a call of communication functions described in section 5.

```
if( strcmp( str, "ACTION_1" ) == 0 )
  {
    /* Put here code for "ACTION_1"  */
    return;
}

  if( strcmp( str, "ACTION_2" ) == 0 )
  {
    /* Put here code for "ACTION_1"  */
    return;
  }
```

If your *ExternalIPC* function performs only one action you are not obliged to use this input argument. The *str* value is one of the input arguments of the *CallExtIPC* call (see MATLAB function list in section 5),

in_arg - pointer to the array containing input arguments. The input arguments are passed to the function by the *CallExtIPC* function. The *ExternalIPC* does not check the size of the *in_arg* array. The user is responsible for passing appropriate number of input arguments,

out_arg - pointer to the array containing output arguments. The output arguments are passed from the *ExternalIPC* function to MATLAB workspace. The maximum size of the *out_arg* array is 4096 double precision floating point elements. The *pd_call* function during the *CallExtIPC* call allocates memory available to the variable returned to MATLAB workspace, so it has to be provided with the information about the size of the returned variable (number of rows and number of columns). This information is contained in the first two elements of the *out_arg* array. The first element is an integer equal to the number of rows. The second element is equal to the number of columns. For instance, if you want to return a matrix with a single row and 10 columns, use the following statements:

```
*out_arg = 1.0;                /* Set number of rows    */
*(out_arg + 1) = 10.0;         /* Set number of columns */
    for( i = 0; i < 10; i++ )
      *(out_arg + 2 + i) = sin( i ); /* Fill the matrix */
```

Be sure to return at least a single element matrix, even if your *ExternalIPC* function does not perform any action:

```
*out_arg = 1.0;
*(out_arg + 1) = 1.0;     /* Single element matrix */
*(out_arg + 2) = -1.0;    /* Return -1.0           */
```

Notes

5. Communication with MATLAB

The communication functions associated with the external DLL library have the following form:

ret = pd_call('FunctionName', [arguments]), where:

ret - is the matrix returned from the function call,

pd_call - is the name of the interface between external DLL library and MATLAB environment,

FunctionName - is the name of the operation performed by the *pd_call* interface. All the operations associated with external DLL are given in the Table 5.1, and described in the following sections. The operation name is passed as MATLAB string variable. The *FunctionName* is not case sensitive. A*rguments* - are optionalarguments required by some actions.

Operation	Description
LoadExtLibrary	Load external DLL library to the memory and: for MATLAB 4 execute the *LibMain* function, for MATLAB 5 execute the statements associated with the *DLL_PROCESS_ATTACH* flag in the *DllMain* function
UnloadExtLibrary	For MATLAB 4 execute the *WEP* finction, For MATLAB 5 execute the statements associated with the *DLL_PROCESS_DETACH* flag in the *DllMain* function and remove the external DLL library from the memory
GetExtAlgName	Return the name of a previously loaded DLL library
CallExtIPC	Execute the *ExternalIPC* procedure
ExtAlgOneStep	Execute the *ExternalController* procedure. The execution is initiated by MATLAB and is performed in *off-line* mode

Table 5.1. External DLL operations.

5.1. FUNCTION LOADEXTALG

***Purpose*:** Load external DLL library to the memory.

***Synopsis*:** *ret = pd_call('LoadExtAlg', 'dll_name')*

***Arguments*:**

dll_name - is the full path and file name of the file containing external DLL library. Typically, the file has the extension DLL. The *dll_name* is a string matrix in MATLAB environment. If only the file name is specified the file is searched in:

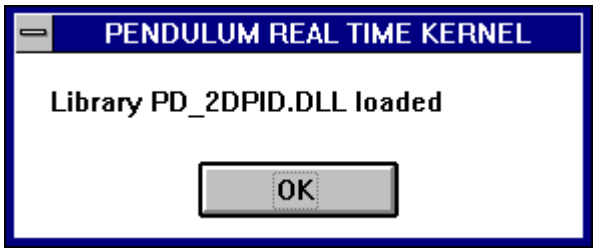
the current directory,

the Windows directory (the directory containing WIN.COM),

the Windows system directory (the directory containing such system files as GDI.EXE),

the directories listed in the PATH environment variable.

***Description*:** This function is called to load a DLL library containing an external controller and an external inter-process communication procedure. The loaded DLL must export the *ExternalController* and *ExternalIPC* functions. After the DLL is loaded the *LibMain* (16-bit DLL) or *DllMain* (32-bit DLL) function is started immediately. The function returns 1 if library is loaded successfully. The window shown below is displayed by RTK. In the case of any fault the error message is displayed in MATLAB command window or in the message box.



The following error messages may be displayed in the MATLAB command window:

LoadExtAlg function requires two arguments - the *LoadExtAlg* function requires two input arguments: the first is function name and the second is the name of external DLL,

LoadExtAlg function requires that the second argument is a string - the external DLL library name must be a string.

After displaying an error message in the MATLAB command window the execution of the function is terminated immediately.

The following messages may appear in the RTK message window:

External library loaded. Unload it first - the external DLL library is already loaded. You must remove the library from the memory first. Use the *UnloadExtAlg* function and next try to load library once more,

Can not load xxxxx library - can not load DLL executable xxxxx. Consider one of the following reasons for the error:

- system is out of memory, executable file is corrupt, or reallocations are invalid,
- file xxxxx is not found,
- path is not found,
- there is a sharing or network-protection error,
- application is designed for a different operating system,
- type of executable file is unknown,
- attempt is made to load a real-mode application developed for an earlier version of Windows,

Can not find ExternalController function - there is no *ExternalController* function exported,

Can not find ExternalIPC function - there is no *ExternalIPC* function exported.

To execute the control algorithm implemented as the* External Controller *function included in the loaded DLL library in real-time, you must set algorithm number in RTK to 99.

For this purpose use the command:

ret = pd_call('SetAlgNo', 99);

Example:

To load pd_2dpid.dll executable located in the D:\MATLAB\DLL directory execute the following command:

ret = pd_call('LoadExtAlg', 'D:\MATLAB\DLL\PD_2DPID.DLL')

5.2. FUNCTION UNLOADEXTALG

***Purpose*:** Remove the external DLL library from the memory.

Synopsis: *ret = pd_call('UnloadExtAlg')*

***Arguments*:** *none*.

Description: This function removes previously aa loaded external DLL from the memory. Before removing it the *WEP* function (16-bit DLL) or the *DllMain* function (32-bit DLL) is executed. If the number of the control algorithm in the RTK was equal to 99 then the number of the control algorithm is set to zero. The function always returns a value of 1.

Example:

To remove the external DLL executable from the memory, execute the following command:

ret = pd_call('UnloadExtAlg')

5.3. FUNCTION GETEXTALGNAME

Purpose: Returns the full path and the name of loaded external DLL executable.

Synopsis: *ret = pd_call('GetExtAlgName')*

Arguments: *none*.

Description: This function returns a string matrix containing the full path and the name of a previously loaded DLL library. The return value is equal to the second argument of the *LoadLibrary* function. An empty matrix is returned if there is no external library loaded or if the library was removed from the memory.

Example:

➢ After executing the command:

ret = pd_call('LoadExtAlg', 'D:\MATLAB\DLL\PD_2DPID.DLL');

➢ the command:

ret = pd_call('GetExtAlgName');

➢ sets the string *D:\MATLAB\DLL\PD_2DPID.DLL* to variable *ret.* After commands:

ret = pd_call('UnloadExtAlg');

ret = pd_call('GetExtAlgName');

➢ the *ret* variable is an empty matrix.

5.4. FUNCTION CALLEXTIPC

Purpose: Executes the *ExternalIPC* procedure.

Synopsis: *ret = pd_call('CallExtIPC', subfunction, input_argument)*

Arguments: ***subfunction*** - string passed to the *ExternalIPC* function as the first argument, usually used to distinguish different actions performed by this function,

input_arguments - data passed to the *ExternalIPC* function as the second argument. Notice: the *ExternalIPC* function does not check the size of *input_argument* matrix. You must pass the appropriate number of elements in the input matrix. A Matrix containing at least one element must be passed as the third input argument.

Description: This function calls the *ExternalIPC* function located in the external DLL library. Two input arguments are passed to the function - one string and one matrix. Before the *ExternalIPC* function is executed the *pd_call* function allocates the array of 4096 double precision elements for output arguments. The output values are returned to MATLAB workspace as the *ret* variable. This function provides a way to exchange data between MATLAB workspace and external DLL library. Typically, this function is used to set parameters of the external controller or to read the value of any variable located in the external DLL module.

Example:

Let us consider the *ExternalIPC* function containing the following C-code:

```
if( strcmp( str, "ACTION_1" ) == 0 )
{
  *out_arg = 1.0;                /* Set number of rows    */
  *(out_arg + 1) = 10.0;         /* Set number of columns */
      for( i = 0; i < 10; i++ )
        *(out_arg + 2 + i) = sin( i ); /* Fill the matrix */
}
```

When you execute the command:

ret = pd_call('CallExtIPC', 'ACTION_1', [1]);

the *ret* variable is the single row vector containing 10 elements. The values of elements are equal to the values of sinus function.

5.5. FUNCTION EXTALGONESTEP

Purpose: Calls *ExternalController* function.

Synopsis: *ret = pd_call('ExtAlgOneStep', input_argument)*

Arguments:

input_arguments - the six-element vector containing:

input_argument(1) - angular position (angle) of the pendulum [rad],

input_argument(2) - angular velocity of the pendulum [rad/sec],

input_argument(3) - position of the cart on the rail [m],

input_argument(4) - velocity of the cart [m/sec],

input_argument(5) - time [sec],

input_argument(6) - desired position of the cart on the rail [m].

Description: This function calls the *ExternalController* function located in the external DLL library. Unlike the *ExternalController* function executed by RTK this function does not call the *ExternalController* at interrupt time. The control value is calculated by the *ExternalController* function according to the value of *input_arguments* and is returned to the MATLAB workspace as *ret* value.

Typically, this function is used for *non-real time* testing of an external control algorithm.

Example:

To obtain and plot a control value for different values of angle and angular velocity execute the following commands:

```
i_ind = 1;
for i = -1 : 0.1 : 1
  j_ind = 1;
  for j = -1 : 0.1 : 1
   surf( i_ind, j_ind ) = pd_call( 'ExtAlgOneStep', [ i j 0 0 0 0
] );
   j_ind = j_ind + 1;
  end
  i_ind = i_ind + 1;
end
mesh( surf );
```

6. Basic Steps to Implement an External Controller

The External Controller was developed to give you the ability to create and apply your own control algorithm for the inverted pendulum system. In this section we present a short summary of steps you must perform when you wish to add your control algorithm. The section is an overview of information given in the previous sections. The applications are given in sections 7,8, 9 and 10.

A user activity of creating its own controller consists of several obligatory and optional steps. Several C-code files must be developed or adapted based on the given template files. The structure of the external DLL is built of two parts: the *ExternalController* and the *ExternalIPC* functions. The *ExternalController* provides a control routine and is accessed at the interrupt time. The *ExternalIPC* consists of gateway routines and performs non real-time communication with the MATLAB environment. Generally, a number of necessary C-code files depends on complexity of the designed controller. For a simple control algorithm one can minimise its effort creating only a source C-file on the basis of the C-code template *pd_tmf.c*. If more complex algorithms are considered, e.g. neural, fuzzy, adaptive etc. then it is recommended to create them as separate files and include into the main source file.

Below are some recommendations which you should follow when developing and executing external DLLs:

- Use the *pd_tmf.c* file as the template file of your external controller,
- notice, that the external DLL is executed when:

  the *LoadExtAlg* function loaded the external DLL successfully, and

  the algorithm number is set to 99 (use the *SetAlgNo* to change the algorithm number),

The DLL file containing an external controller must be located in the MATLAB root directory. Execute the* matlabroot *command in the MATLAB Command Window to find out the MATLAB root directory. Remember, that during experiments with external DLLs the MATLAB current directory must be this same as the MATLAB root directory. Use the MATLAB command pwd to check the name of the current directory and the* cd *to change it.

6.1. CREATING THE EXTERNALCONTROLLER FUNCTION

The *ExternalController* function is a dedicated controller for the inverted pendulum system. Knowledge of its architecture and variation intervals is essential to build a correct algorithm. The key to the successful implementation of such an algorithm is flexibility subject only to being limited to:

Being able to perform all the calculations necessary for the control algorithms, within the time allowed by the the real-time sampling period (0.001 [s] - minimal value for RTK)

Having a fixed I/O structure - 8 inputs (one 20-elements input vector of parameters, seven single inputs) and 1 output of an external controller.

This allows a great deal of functionality to be shoehorned into one short time step.

Apart from the I/O architecture, the following states and normalised values and limits of the control (usually exported from the MATLAB environment or stored in the DLL library), can be stored in the *pd_tmf.c* External DLL library template file/

- the angle position value x_2 normalised to be in the interval from $[-\pi \div +\pi]$,
- the static cart friction f_c compensated. The absolute value of control is enlarged by the static friction. The relation between a new compensated control and an old non-compensated control has the form: $u_{new} = \text{sign}(u_{old})\,(|u_{old}| + f_c)$,
- the cart position x_1 constrained by the rail length x_{1max}=0.5[m], $x_1 \in [-x_{1max}, x_{1max}]$,
- the admissible normalised control (steering force u) bounded $|u(t)| \leq 1$.

6.2. CREATING THE EXTERNALIPC FUNCTION

The I/O structure of the *ExternalIPC* function is shown in section 4.2. Notice, that the *ExternalIPC* function does not check the size of the input arguments. Therefore, you must remember not to exceed the limits of input variables.

The *SetP* and *GetP* functions from Reference Manual implement the simple inter-process communication between the MATLAB workspace and the RTK. These functions allow the user to read/write from/to the array containing 20 numbers (see Reference Manual for the detailed description of the *SetP* and *GetP* functions). This array is located in the RTK workspace.

The implementation of the* ExternalIPC *function is necessary only in the case when a more complex data exchange protocol is required.

6.3. TESTING IN THE MATLAB ENVIRONMENT

The *ExternalController* operates in real-time (excited by the RTK timer) or in non real-time mode (excited from MATLAB environment). Non-real time mode is used for testing and debugging. The *ExternalIPC* establishes exchange of information between the workspace of MATLAB and the *ExternalController* in both modes. Fig. 6.1 illustrates how to communicate with the external DLL file.

TESTING MODE

```
ret = pd_call('LoadLibrary')                          % RTK function
ret = pd_call('LoadExtAlg', put dll.name here)        % External
function
ret = pd_call('CallExtIPC','SetParam',[.....])        % External
function

% initialization of testing loop
Control=pd_call('ExtAlgOneStep',[put input arguments: 6 elements
                                 vector])

ret = pd_call('UnloadExtAlg')                         % External
function
```

OPERATING MODE

```
ret = pd_call('LoadLibrary')                          % RTK function
ret = pd_call('LoadExtAlg', put dll.name here)        % External
    function
ret = pd_call('CallExtIPC','SetParam',[.....])        % External
function
ret =pd_call('SetAlgNo',99)                           % RTK function
% initialization of data acquisition
while pd_call('GetNoOfSamples')<200                   % RTK function
  ;
end
h=pd_call('GetHistory');                              % RTK function
plot(h);                                              % MATLAB
ret = pd_call('UnloadExtAlg')                         % External
function
```

Fig.6.1 Testing and operating mode.

Notes

7. Example 1: Non-linear Stabilising Controller with Tuned Coefficients

Let us implement a controller to stabilise the pendulum near to the upright equilibrium point. The assumed control law is:

$$U = \left(1 + \sqrt{\sum_{i=1}^{4} D_i x_i^2}\right) \sum_{j=1}^{4} K_j x_j + friction_compensation ,$$

$$friction_compensation = U + sign(U) f_c ,$$

where:

U - control value for the DC drive,

x_1 - error between the desired and current position of the cart,

x_2 - angular position of the pendulum,

x_3 - velocity of the cart,

x_4 - angular velocity of the pendulum,

$K_1 \ldots K_4$ - feedback coefficients,

$D_1 \ldots D_4$ - the coefficients of the distance to the equilibrium point,

f_c - friction compensation constant.

All parameters of the controller will be passed as the first argument of the *ExternalController* function.

The C-code source file is shown below (see *pd_ss.c* file on the installation disk). Notice, that the *ExternalIPC* function is not performing any operation.

```
#include <windows.h>
#include <stdio.h>
#include <string.h>
#include <dos.h>
#include <math.h>

#define PI_CONST  ( 3.14159265358979323846 )

#ifdef WIN32
```

```
  #define DLLEXPORT __declspec(dllexport)
  HINSTANCE    hInstDLL;
#endif


#ifdef WIN32   // 32-bit DLL

BOOL WINAPI DllMain( HINSTANCE hDLLInst, DWORD fdwReason,
                     LPVOID lpvReserved )
{
  char msg_str[ 150 ];
  static char ModuleName[ 128 ];
    switch (fdwReason)
        {
        case DLL_PROCESS_ATTACH:
            hInstDLL = hDLLInst;
            GetModuleFileName( hDLLInst, ModuleName,
                               sizeof( ModuleName ) - 30 );
            sprintf( msg_str, "External DLL Library: %s",
ModuleName );
            MessageBox( (HWND)NULL,
                "Entry point to external DLL reached",
                 msg_str, MB_OK );
            break;
        case DLL_PROCESS_DETACH:
            sprintf( msg_str, "External DLL Library: %s",
ModuleName );
            MessageBox( (HWND)NULL,
                "Exit point of external DLL reached",
                 msg_str, MB_OK );
            break;
        case DLL_THREAD_ATTACH:
            break;
```

```
        case DLL_THREAD_DETACH:
            break;
        }
    return TRUE;
}

#else    // 16-bit DLL

int FAR PASCAL LibMain( HINSTANCE hInstance, WORD wDataSegment,
                        WORD wHeapSize, LPSTR lpszCmdLine )
{
  char msg_str[ 128 ];

  sprintf( msg_str, "External DLL Library: %s", __FILE__ );
  MessageBox( (HWND)NULL, "Entry point to external DLL reached",
              msg_str, MB_OK );

  return 1;   /* Indicate that the DLL was initialized
successfully */
}

int FAR PASCAL WEP ( int bSystemExit )
{
  char msg_str[ 128 ];

  sprintf( msg_str, "External DLL Library: %s", __FILE__ );
  MessageBox( (HWND)NULL, "Exit point of external DLL reached",
              msg_str, MB_OK );

  return 1;
}

#endif
```

```
#ifdef WIN32
DLLEXPORT void __stdcall ExternalController(
      double *Param,
      double PendPos,
      double PendVelocity,
      double CartPos,
      double CartVelocity,
      double Time,
      double DesValue,
      double *Control )
#else
void FAR PASCAL ExternalController(
      double far *Param,        /* Parameters of the controller
*/
      double far  PendPos,      /* Angle [rad]
*/
      double far  PendVelocity, /* Angular velocity [rad/s]
*/
      double far  CartPos,      /* Cart's position [m]
*/
      double far  CartVelocity, /* Cart's velocity [m/s]
*/
      double far  Time,         /* Time [s]
*/
      double far  DesValue,     /* Desired value of the cart [m]
*/
      double far *Control )     /* Calculated control value
*/
#endif
{
  static double ctrl = 0.0;
  static double x1, x2, x3, x4;
  static double fc, K1, K2, K3, K4, D1, D2, D3, D4;
  static double distance;
```

```
/* Parameters */
fc = Param[ 0 ];    /* Friction compensation */
K1 = Param[ 1 ];    /* K1                    */
K2 = Param[ 2 ];    /* K2                    */
K3 = Param[ 3 ];    /* K3                    */
K4 = Param[ 4 ];    /* K4                    */
D1 = Param[ 5 ];    /* D1                    */
D2 = Param[ 6 ];    /* D2                    */
D3 = Param[ 7 ];    /* D3                    */
D4 = Param[ 8 ];    /* D4                    */

/* Auxiliary statements */
x1 = CartPos - DesValue;  /* Error of the cart position */
x2 = PendPos;
x3 = CartVelocity;
x4 = PendVelocity;

/* Normalization of pendulum angle           */
/* If angle out of range from -pi to +pi */
while( x2 >  PI_CONST )  x2 -= 2*PI_CONST;
while( x2 < -PI_CONST )  x2 += 2*PI_CONST;

distance = sqrt( D1*x1*x1 + D2*x2*x2 + D3*x3*x3 + D4*x4*x4 );
ctrl = ( 1.0 + distance) * ( K1*x1 + K2*x2 + K3*x3 + K4*x4 );

/* Friction compensation */
if ( ctrl > 0.0 ) ctrl += fc;
if ( ctrl < 0.0 ) ctrl -= fc;

/* Check limits of the control value */
if(ctrl <= -1.0 ) ctrl = -1.0;
if(ctrl >= +1.0 ) ctrl = +1.0;
```

```
  /* Return control value */
  *Control = ctrl;
}

#ifdef WIN32
DLLEXPORT void __stdcall ExternalIPC(
         char   *str,
         double *in_arg,
         double *out_arg )
#else
void FAR PASCAL ExternalIPC(
        char   far *str,           /* Function          */
        double far *in_arg,        /* Input arguments   */
        double far *out_arg )      /* Output values     */
#endif
{
   *( out_arg + 0 ) = 1;  *( out_arg + 1 ) = 1;
   *( out_arg + 2 ) = -1;  /* Always returns -1.0 */
}
```

Now, you can compile your DLL library. Next, start the experiment by executing the following commands:

```
ret= pd_call('loadextalg', 'pd_ss.dll');
ret= pd_call('setp', [ ..... ] ); % Insert 9 parameters here
ret= pd_call('setalgno', 99 );
```

8. Example 2: Off-line Testing of *Externalcontroller*

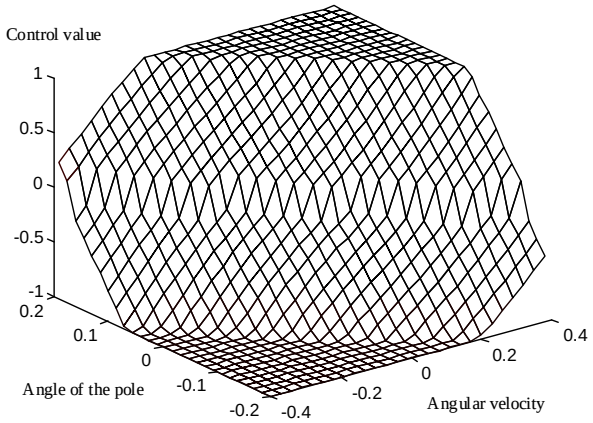
The following set of commands loads the external DLL library *pd_ss.dll* , sets parameters and calculates the value of control on the angle vs. angular velocity plane. The C-source code of the DLL library is shown in the Example 1.

```
a= pd_call('loadlibrary');

a= pd_call('loadextalg', 'pd_ss.dll');
name = pd_call('getextalgname');
disp( [ 'Name: ' name ] );

p = pd_call( 'getp' );
p( 1 : 9 ) = [ 0.1 0.8 6.0 1.4 2.6 1.0 1.0 1.0 1.0 ];
ret = pd_call( 'setp', p );

surf = [];
i_ind = 1;
for i = -0.2 : 0.015 : 0.2
  j_ind = 1;
  for j = -0.4 : 0.03 : 0.4
   surf( i_ind, j_ind ) = pd_call( 'ExtAlgOneStep',[ i j 0 0 0 0
]);
    j_ind = j_ind + 1;
  end
  i_ind = i_ind + 1
end

a= pd_call('unloadextalg');
```

To plot the calculated surface execute the following commands:

```
y = -0.2 : 0.015 : 0.2;
x = -0.4 : 0.03 : 0.4;
```

```
mesh( x, y, surf );
```


Control value
1
0.5
0
-0.5
-1
0.2
0.1
0
-0.1
-0.2
Angle of the pole
-0.4
-0.2
0
0.2
0.4
Angular velocity

9. Example 3: 2-D Pid Controller

We will implement the controller shown in figure 8.1. There are two PID controllers. The first one controls the angular position of the pendulum. The second is used to control the position of the cart. The outputs of the PID controllers are added to produce the final control value for the D/A converter. Parameters of the controllers are exchanged using the *ExternalIPC* function.

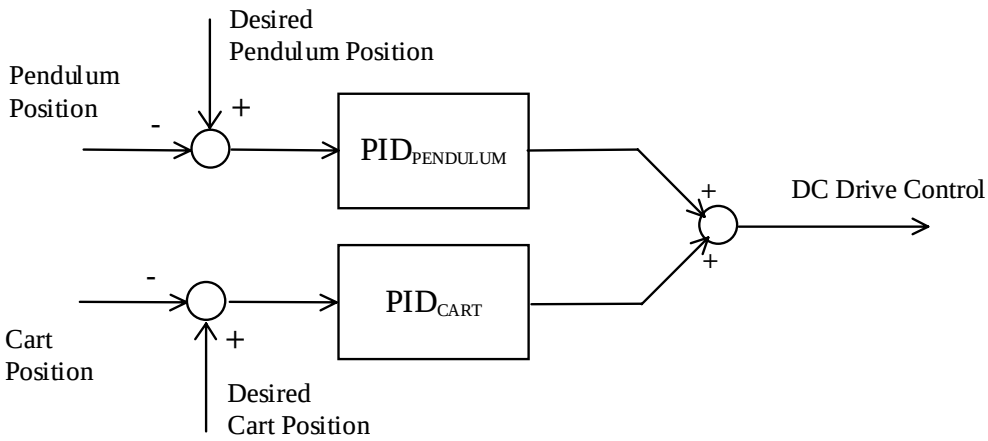


Fig.9.1. Structure of the PID controller.

The controller is described by the following equations.

The proportional part:

$e_\alpha = r_\alpha - \alpha$,

$e_s = r_s - s$,

where:

e_α, e_s - errors of pendulum and cart positions,

r_α, r_s - reference values of pendulum and cart positions,

α, s - current values of pendulum and cart positions.

The integrators are described as follows:

$$I_{PENDULUM}(t) = K_{I_PENDULUM} \int_0^t e_\alpha dt ,$$

$$if\,(I_{PENDULUM} > I_{PENDULUM_SAT})\,then\,I_{PENDULUM} = I_{PENDULUM_SAT} ,$$

$$if\,(I_{PENDULUM} < -I_{PENDULUM_SAT})\,then\,I_{PENDULUM} = -I_{PENDULUM_SAT} ,$$

$$I_{CART}(t) = K_{I_CART} \int_0^t e_S dt ,$$

$$if\,(I_{CART} > I_{CART_SAT})\,then\,I_{CART} = I_{CART_SAT} ,$$

$$if\,(I_{CART} < -I_{CART_SAT})\,then\,I_{CART} = -I_{CART_SAT} ,$$

where:

$K_{I_PENDULUM}, K_{I_CART}$ - are gains of the I parts,

$I_{PENDULUM_SAT}, I_{CART_SAT}$ - are the saturation values of the integrators.

The outputs from the $PID_{PENDULUM}$ and PID_{CART} parts are equal to:

$$U_{PENDULUM} = K_{P_PENDULUM} e_\alpha + I_{PENDULUM}(t) + K_{D_PENDULUM} \frac{de_\alpha}{dt} ,$$

$$U_{CART} = K_{P_CART} e_s + I_{CART}(t) + K_{D_CART} \frac{de_s}{dt} ,$$

$K_{P_PENDULUM}, K_{D_PENDULUM}, K_{P_CART}, K_{D_CART}$ - are parameters of the proportional and derivative parts of the controller respectively.

Finally, the control for DC drive is:

$$U = U_{PENDULUM} + U_{CART} ,$$

$$if\,(U < -U_{MAX})\,then\,U = -U_{MAX} ,$$

$$if\,(U > U_{MAX})\,then\,U = U_{MAX} ,$$

where:

U_{MAX} - is maximum value of the control.

The C-code source file is shown below (see the *pd_2dpid.c* file on the installation disk).

```
#include <windows.h>
#include <stdio.h>
#include <string.h>
#include <dos.h>
#include <math.h>


#define IPC_GetParam  "GetParam"  /* Read parameters - 12 values
*/
#define IPC_SetParam  "SetParam"  /* Set parameters
*/

#define PI_CONST  ( 3.14159265358979323846 )

#define IPC_GetParam  "GetParam"   /* Read parameters - 12 values
*/
#define IPC_SetParam  "SetParam"   /* Set parameters
*/

#ifdef WIN32
  #define DLLEXPORT __declspec(dllexport)
  HINSTANCE    hInstDLL;
#endif

double KpAngle, KiAngle, KdAngle;
double KpPosition, KiPosition, KdPosition;
double IMaxAngle, IMaxPosition;
double UMax;
double TSample;
double RefAngle, RefPosition;
```

```
char ModuleName[ 128 ];


#ifdef WIN32   // 32-bit DLL

BOOL WINAPI DllMain( HINSTANCE hDLLInst, DWORD fdwReason,
                     LPVOID lpvReserved )
{
  char msg_str[ 150 ];
  static char ModuleName[ 128 ];
    switch (fdwReason)
        {
        case DLL_PROCESS_ATTACH:
            hInstDLL = hDLLInst;

            KpAngle = KiAngle = KdAngle = 0.0;
            KpPosition = KiPosition = KdPosition = 0.0;
            IMaxAngle  = IMaxPosition = 0.0;
            UMax     = 0.0;
            TSample  = 0.001;
            RefAngle = RefPosition = 0.0;

            GetModuleFileName( hDLLInst, ModuleName,
                 sizeof( ModuleName ) - 30 );
            sprintf( msg_str, "External DLL Library: %s",
                 ModuleName );
            MessageBox( (HWND)NULL,
                        "Entry point to external DLL reached",
                        msg_str, MB_OK );
            break;

        case DLL_PROCESS_DETACH:
            sprintf( msg_str, "External DLL Library: %s",
```

```
                ModuleName );
            MessageBox( (HWND)NULL,
                        "Exit point of external DLL reached",
                        msg_str, MB_OK );
            break;

        case DLL_THREAD_ATTACH:
            break;

        case DLL_THREAD_DETACH:
            break;
        }
    return TRUE;
}

#else    // 16-bit DLL

int FAR PASCAL LibMain( HINSTANCE hInst, WORD wDataSegment,
                        WORD wHeapSize, LPSTR lpszCmdLine )
{
  char msg_str[ 150 ];

  GetModuleFileName( hInst, ModuleName, sizeof(ModuleName) - 30 );

  sprintf( msg_str, "External DLL Library: %s", ModuleName );
  MessageBox( (HWND)NULL, "Entry point to external DLL reached",
              msg_str, MB_OK );

  KpAngle = KiAngle = KdAngle = 0.0;
  KpPosition = KiPosition = KdPosition = 0.0;
  IMaxAngle  = IMaxPosition = 0.0;
  UMax     = 0.0;
  TSample  = 0.001;                /* Min sample period == 1ms */
```

```
  RefAngle = RefPosition = 0.0;    /* Upright pos. of the pend.*/

  return 1;   /* The DLL was initialized successfully */
}

int FAR PASCAL WEP ( int bSystemExit )
{
  char msg_str[ 150 ];
  sprintf( msg_str, "External DLL Library: %s", ModuleName );
  MessageBox( (HWND)NULL, "Exit point of external DLL reached",
              msg_str, MB_OK );

  return 1;
}

#endif


#ifdef WIN32
DLLEXPORT void __stdcall ExternalController(
     double *Param,
     double PendPos,
     double PendVelocity,
     double CartPos,
     double CartVelocity,
     double Time,
     double DesValue,
     double *Control )
#else
void FAR PASCAL ExternalController(
     double far *Param,        /* Parameters of the controller  */
     double far  PendPos,      /* Angle [rad]                   */
     double far  PendVelocity, /* Angular velocity [rad/s]      */
```

```
    double far  CartPos,       /* Cart's position [m]           */
    double far  CartVelocity, /* Cart's velocity [m/s]         */
    double far  Time,          /* Time [s]                      */
    double far  DesValue,      /* Desired value of the cart [m] */
    double far *Control )      /* Calculated control value      */
#endif
{
  static double UAngle, UPosition, U;
  static double ErrAngle, ErrPosition;
  static double ErrAnglePrev = 0.0, ErrPositionPrev = 0.0;
  static double IAngle = 0.0, IPosition = 0.0;

  ErrAngle    = RefAngle - PendPos;
  ErrPosition = RefPosition - CartPos;

  /* Normalization of the ErrAngle to the range from -pi to +pi */
  while( ErrAngle >  M_PI )  ErrAngle -= 2*M_PI;
  while( ErrAngle < -M_PI )  ErrAngle += 2*M_PI;


  /* -----> PID Angle <----- */
  /* Calculate the IAngle part */
  if ( KiAngle == 0.0 )
    IAngle = 0.0;  /* Set the I part to zero if KiAngle == 0 */
  else
    IAngle += KiAngle * ErrAngle * TSample;
  /* Saturation of the IAngle part */
  if ( IMaxAngle > 0.0 )
  {
    if ( IAngle >  IMaxAngle )   IAngle =  IMaxAngle;
    if ( IAngle < -IMaxAngle )   IAngle = -IMaxAngle;
  }
  /* Add angle PID parts: PID = P + I + D */
```

```
  UAngle = KpAngle*ErrAngle + IAngle +
      KdAngle*(ErrAngle-ErrAnglePrev)/TSample;

  /* -----> PID Position <----- */
  /* Calculate the IPosition part */
  if ( KiPosition == 0.0 )
    IPosition = 0.0;  /* Set the I part to zero if KiPosition == 0
*/
  else
    IPosition += KiPosition * ErrPosition * TSample;
  /* Saturation of the IPosition part */
  if ( IMaxPosition > 0.0 )
  {
    if ( IPosition >  IMaxPosition )   IPosition =  IMaxPosition;
    if ( IPosition < -IMaxPosition )   IPosition = -IMaxPosition;
  }
  /* Add Position PID parts: PID = P + I + D */
  UPosition = KpPosition*ErrPosition + IPosition +
      KdPosition*(ErrPosition-ErrPositionPrev)/TSample;

  /* Control value for the DC drive */
  U = UAngle + UPosition;

  ErrAnglePrev    = ErrAngle;
  ErrPositionPrev = ErrPosition;

  /* Control limits */
  if( U >  fabs( UMax ) )  U =  fabs( UMax );
  if( U < -fabs( UMax ) )  U = -fabs( UMax );

  /* Check limits of the control value */
  if( U <= -1.0 ) U = -1.0;
  if( U >= +1.0 ) U = +1.0;
```

```
  *Control =  U;
}


#ifdef WIN32
DLLEXPORT void __stdcall ExternalIPC(
        char   *str,
        double *in_arg,
        double *out_arg )
#else
void FAR PASCAL ExternalIPC(
        char   far *str,       /* Function          */
        double far *in_arg,    /* Input arguments   */
        double far *out_arg )  /* Output values     */
#endif
{
   char function[ 128 ];

   strcpy( function, str );
   strupr( function );       /*  Do not care cases  */

if( strcmp( function, strupr( IPC_SetParam ) ) == 0 )
   {
     KpAngle      = *( in_arg +  0 );
     KiAngle      = *( in_arg +  1 );
     KdAngle      = *( in_arg +  2 );
     KpPosition   = *( in_arg +  3 );
     KiPosition   = *( in_arg +  4 );
     KdPosition   = *( in_arg +  5 );
     IMaxAngle    = *( in_arg +  6 );
     IMaxPosition = *( in_arg +  7 );
     UMax         = *( in_arg +  8 );
```

```
      TSample       = *( in_arg +  9 );
      RefAngle      = *( in_arg + 10 );
      RefPosition   = *( in_arg + 11 );

      *( out_arg + 0 ) = 1;  *( out_arg + 1 ) = 1;
      *( out_arg + 2 ) = 1;
      return;
    }

    if( strcmp( function, strupr( IPC_GetParam ) ) == 0 )
    {
      *( out_arg +  0 ) = 1;  *( out_arg + 1 ) = 12;
      *( out_arg +  2 ) = KpAngle;
      *( out_arg +  3 ) = KiAngle;
      *( out_arg +  4 ) = KdAngle;
      *( out_arg +  5 ) = KpPosition;
      *( out_arg +  6 ) = KiPosition;
      *( out_arg +  7 ) = KdPosition;
      *( out_arg +  8 ) = IMaxAngle;
      *( out_arg +  9 ) = IMaxPosition;
      *( out_arg + 10 ) = UMax;
      *( out_arg + 11 ) = TSample;
      *( out_arg + 12 ) = RefAngle;
      *( out_arg + 13 ) = RefPosition;
      return;
    }

    *( out_arg + 0 ) = 1;  *( out_arg + 1 ) = 1;
    *( out_arg + 2 ) = -1;
}
```

Notice, that:

in the *ExternalController* we do not use the *Param* input argument. All parameters are passed by the *ExternalIPC* function. You must set the parameters of the controller first and start the controller next. Use the following commands:

```
pd_call( 'LoadLibrary' );
pd_call( 'LoadExtAlg', DLL_name );
par = [ your 12 parameter values ];
pd_call( 'CallExtIPC', 'SetParam', par );
pd_call( 'SetAlgNo', 99 );
```

the controller uses the desired values for the position of the cart and for the angle of the pendulum. If the desired value of the angle is equal to zero the controller stabilises the pendulum in the upright position. If the desired value is equal to $\pm\pi$ the cart-pendulum system works as a crane.

Notice, that only two values for the pendulum's desired value are admissible: 0 and $\pm\pi$.

Notes

10. Example 4: Simulink Model for the 2-D Pid Controller

There are two Simulink models available in MATLAB 5 - one for stabilising the pendulum in the down position (crane mode) and the other for stabilising the pendulum in the upright position (inverted pendulum mode). In both cases the controller described in the previous section is used. It is obvious that the crane mode and the inverted pendulum mode require different sets of controller parameters. A few different parameter sets were prepared for both control modes and they are available in the Simulink models.

The names of the Simulink models for the crane mode and inverted pendulum mode are *pd_2ddn* and *pd_2dup* respectively. They are stored in appropriate files on the installation diskette. They files containing Simulink models have extension *mdl*.

10.1. CRANE MODE SIMULINK MODEL

The Simulink model for monitoring of the experiments with the *pd_2dpid.dll* library operating in the crane mode is shown in Fig. 10.1. The main function of this model is not to simulate the system but to transfer data to and from the Real Time Kernel. This is the main difference between the model and a typical model in Simulink.

The main block of the presented Simulink model is *Real Time Task*. It contains the S-function block calling the *pd_2dsf.m* S-function (see Fig. 10.2). The *Real Time Task* is preceded by the signal generators, which act as references sources for the cart position. The output of the *Real Time Task* is a vector which contains information about the pendulum angle and angular velocity, the cart position, velocity and reference position, and the control value for the DC drive. The values of the output vector can be shown on Simulink scopes and are transferred to MATLAB workspace (*To Workspace* block) as the *hist* variable. The *hist* matrix contains additionally the time vector (*Clock* block).

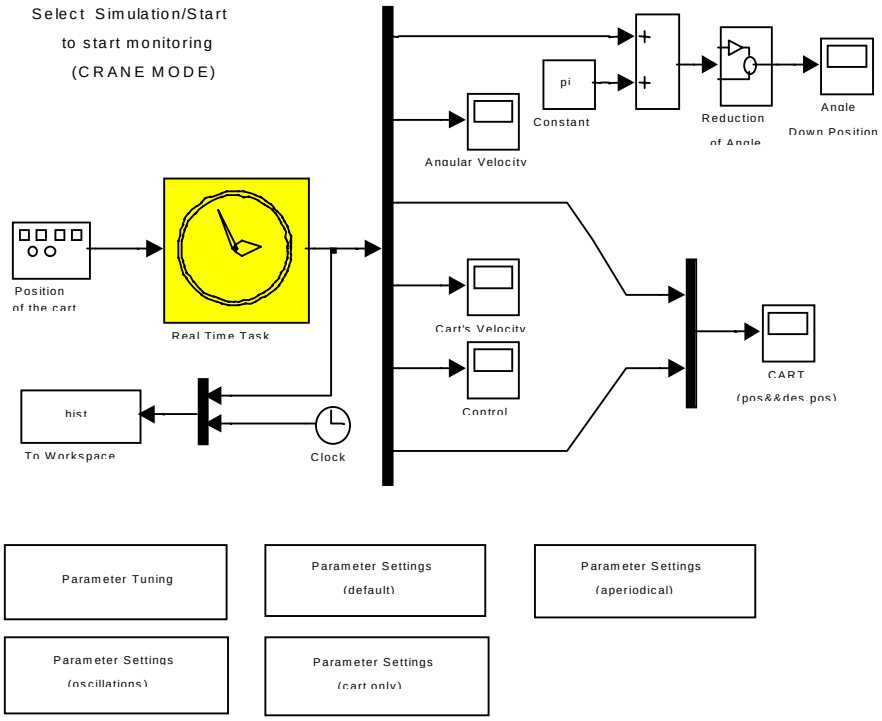


Fig. 10.1. Simulink model for the crane mode (*pd_2ddn.m* file).

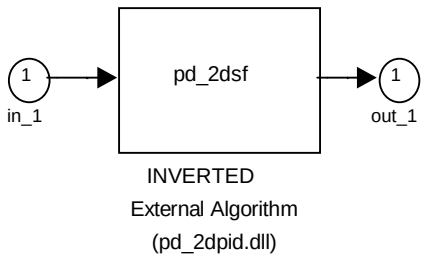


Fig. 10.2. The *Real-Time Task* block.

The *pd_2dsf.m* file contains the S-function associated with the *Real Time Task* block. The function has one parameter: down sampling ratio. This parameter decreases the rate of output data flow from the output of the *Real Time Task* (see Fig. 10.3). For example, if the downsampling coefficient is 10, for each ten sampling periods only one changes the output of the block.

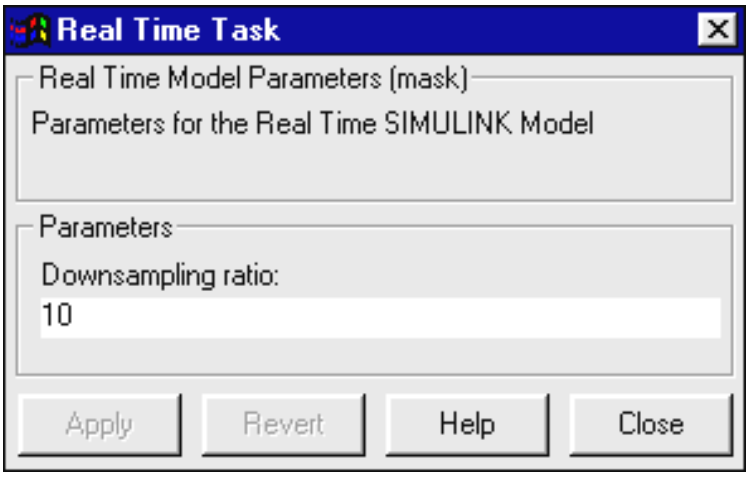


Fig. 10.3. Parameters of the *Real-Time Task* block.

The S-function block included in the *Real Time Task* block contains two parameters: the S-function file name (*pd_2dsf)* and the downsampling coefficient value (*ds*) (see Fig. 10.4).

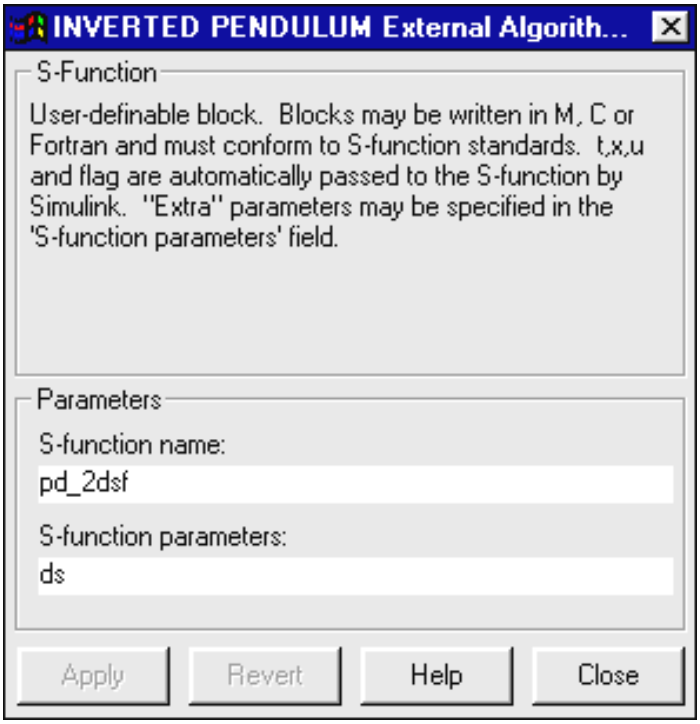


Fig. 10.4. Parameters of the *pd_2dsf* S-function block.

The name of the *Real Time Task* block parameter is set during masking of the *Real-Time Task* block. See the *Initialisation commands* field in Fig. 10.5.

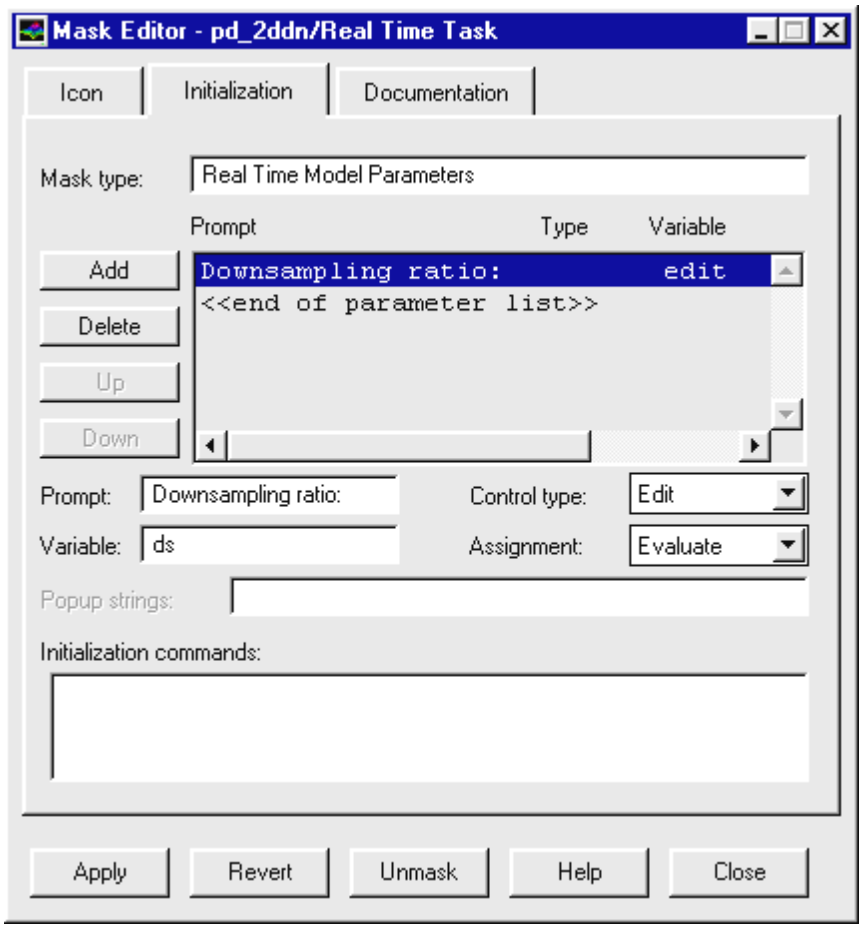


Fig. 10.5. Mask definition for the *Real-Time Task* block.

The blocks *Parameter Settings (default)*, *Parameter Settings (aperiodical)*, *Parameter Settings (oscillations)*, *Parameter Settings (cart only)* and *Parameter Tuning* are used to choose different sets of the controller parameters. Double clicking the mouse over one of the first four blocks mentioned above changes the behaviour of the system during executing the Simulink model. The values of the parameter sets are shown in Table 10.1.

The *Parameter Tuning* block can be used to change parameters by using sliders (see Fig. 10.6). See the *pd_2dsl.m* file for implementation details of Fig. 10.6.

Table 10.1.

	Parameter Settings (default)	*Parameter Settings (aperiodical)*	*Parameter Settings (oscillations)*	*Parameter Settings (cart only)*
KpAngle	7.0	5.0	6.0	0.0
KiAngle	0.0	0.0	0.0	0.0
KdAngle	0.0	0.0	0.0	0.0
KpPosition	10.0	10.0	16.0	10.0
KiPosition	2.8	1.0	2.0	1.0
KdPosition	3.0	5.0	1.0	5.0
IMaxAngle	0.8	0.8	0.8	0.8
IMaxPosition	0.8	0.8	0.8	0.8
UMax	0.8	0.8	0.8	0.8
TSample	0.01	0.01	0.01	0.01
RefAngle	pi	pi	pi	pi
RefPosition	0.0	0.0	0.0	0.0

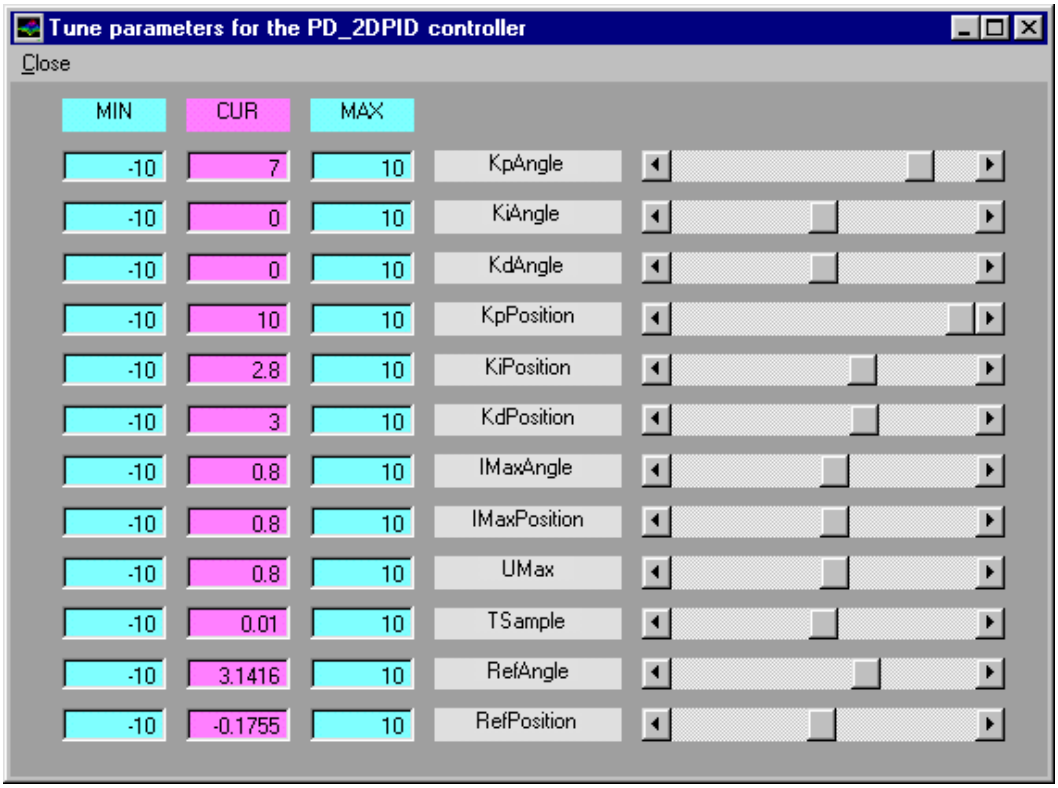
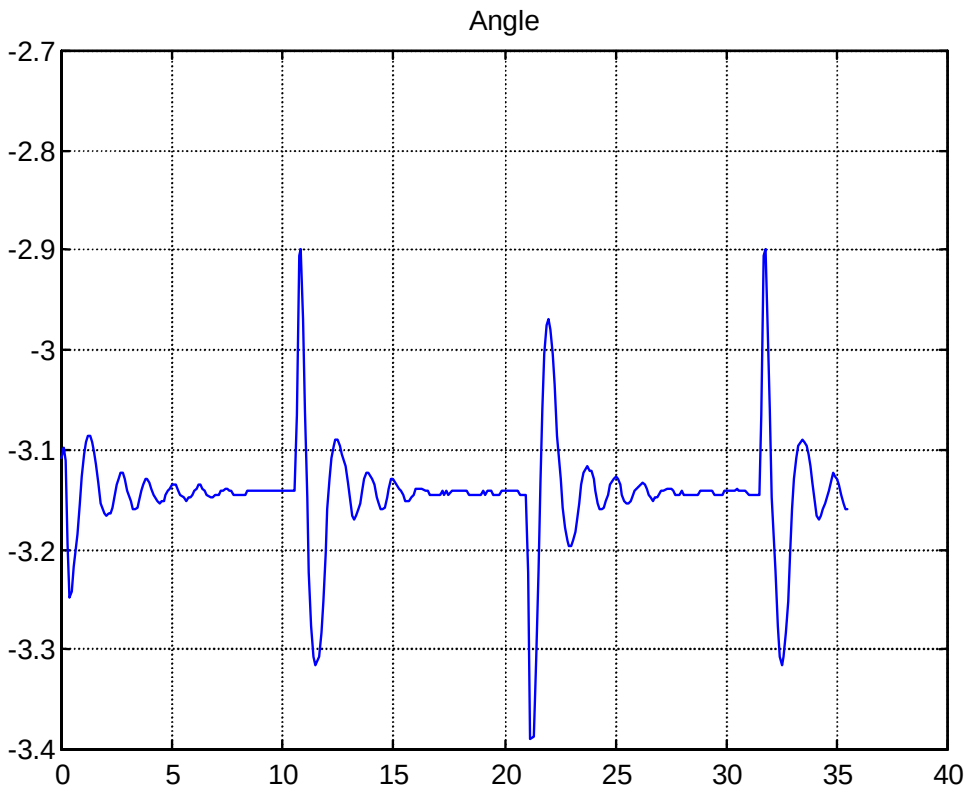
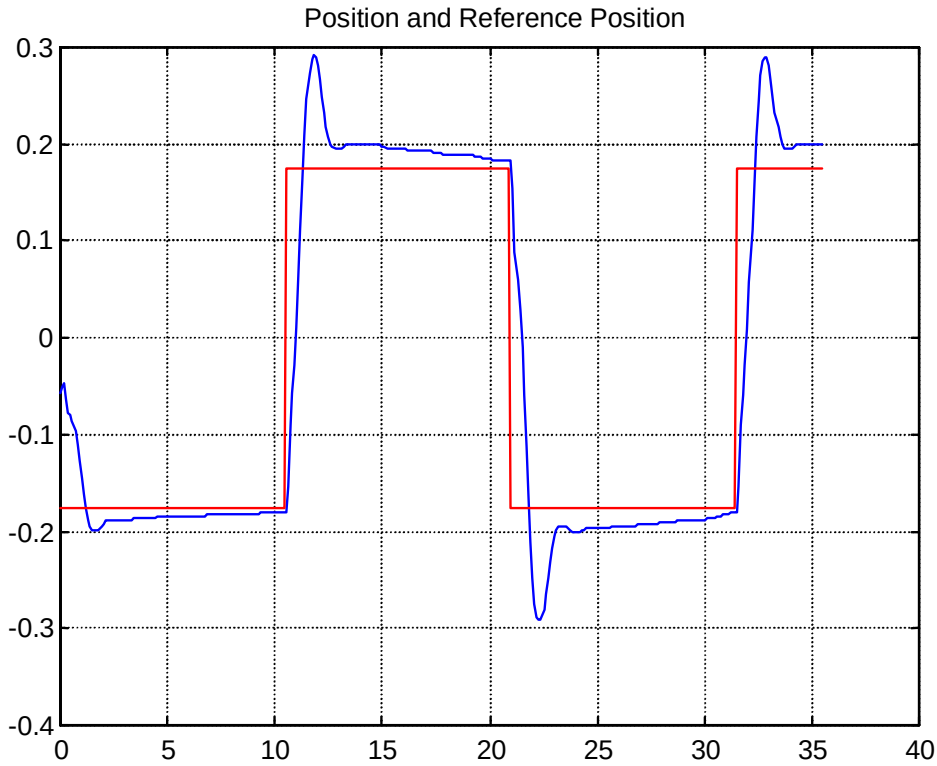


Fig. 10.6. Tuning parameters of the *pd_2dpid* controller.

The history of the experiment is transferred to the MATLAB workspace as the *hist* experiment. The statements below can be used to display the history of the experiment. The parameters were changed during data collection and you can notice different system behaviour at the beginning and at the end of the experiment.

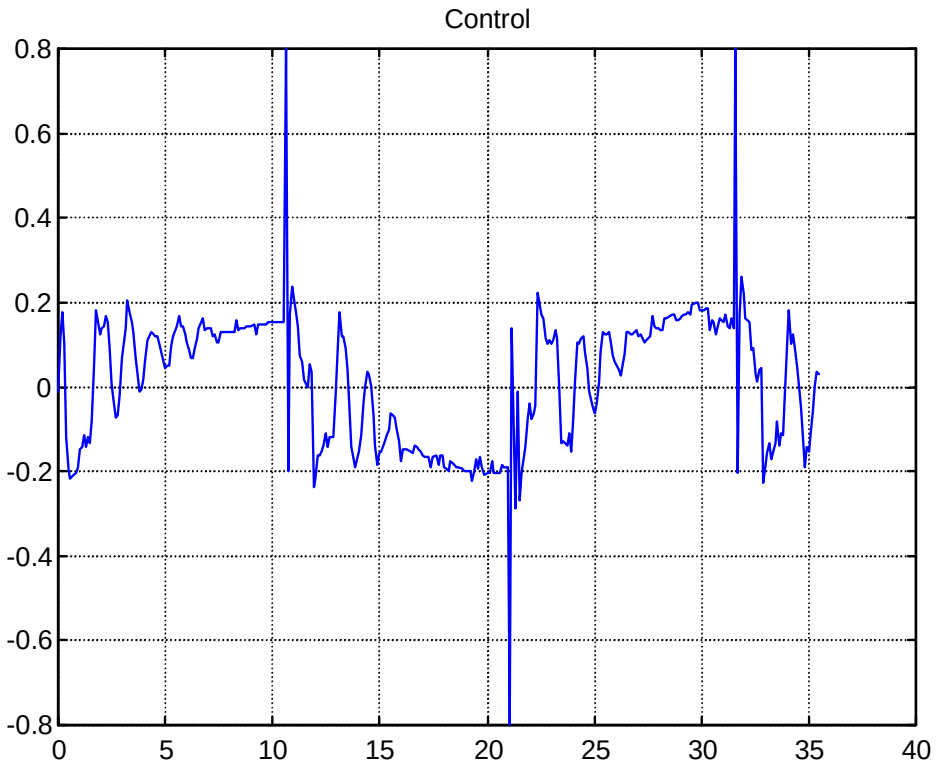


plot(hist(:, 7), hist(:, 1)); grid; title('Angle')



plot(hist(:, 7), hist(:, 3), hist(:, 7), hist(:, 6));

title('Position and Reference Position')



plot(hist(:, 7), hist(:, 5)); grid; title('Control')

The "body" of the *pd_2dsf.m* S-function file is shown below. Notice that the S-function uses the *flag* input argument in a different way from the usual. The flag equal to 0 is used to set the size of the model (the number of continuous states, the number of discrete states, the number of outputs and the number of inputs), to load the external library containing the controller, to set default parameters of the controller, to set the sample period, to start the experiment (set algorithm number to 99) and to set initial value of the auxiliary variables used in the S-function. The flag equal to 2 is used to set a new value of reference position of the cart. The value of the reference position is obtained from the signal generator *Position of the cart* (Fig. 10.1) and is passed to the S-function as the input argument *u*. The flag equal to 3 is used to read the history of the experiment from the Real-Time Kernel and transfer it to the output of the *Real Time Task* block. The flag equal to 4 is used to calculate the next discrete time point. This flag is used to synchronise the time of the experiment and the time of Simulink model. The flag equal to 9 is used to terminate the experiment. The algorithm number is set to zero and the external DLL controller is removed from the memory.

```
function [ sys, x0, str, ts ] = sfunc( t, x, u, flag, downsamp)
%
% S-function pd_2dsf.m
%
%
% Parameters:
%    t         - time,
%    x         - state vector,
%    u         - input to the S-function block,
%    flag      - the value passed to S-function by Simulink to
distinguish
%                different actions. The arguments t, x, u and flag
are set
%                and passed to S-function by Simulink
automatically,
%    downsamp - downsapling ratio. Defines how many samples is
transfered
%                to the output of the S-function block. If dawnsamp
is equel
%                to 10 only 1 sample of every 10 samples is
transfered from
%                Real Time Task to the output of the S-function
block,
%



global history


%
% The following outlines the general structure of an S-function.
%
switch flag,


  case 0,     % Initialization
    % call simsizes for a sizes structure, fill it in and convert
it to a
    % sizes array
    sizes = simsizes;
```

```
    % Set number of continuous states, number of discrete states,
number of
    % outputs and number of inputs.
    % 0 continuous states, 1 discrete state, 6 outputs, 1 inputs
    sizes.NumContStates  = 0;
    sizes.NumDiscStates  = 1;
    sizes.NumOutputs     = 6;
    sizes.NumInputs      = 1;
    sizes.DirFeedthrough = 0;
    sizes.NumSampleTimes = 1;     % at least one sample time is
needed

    sys = simsizes(sizes);

    % Load external library PD_2DPID.DLL
    lib_name = 'PD_2DPID.DLL';
     l_ln = pd_call( 'GetExtAlgName' );
    if isempty( l_ln ) == 1
       dummy = pd_call( 'LoadExtAlg', lib_name );
    else
      if length( l_ln ) ~= length( lib_name )
         dummy = pd_call( 'UnloadExtAlg' );
         dummy = pd_call( 'LoadExtAlg', lib_name );
      else
        if min( l_ln == lib_name ) == 0
           dummy = pd_call( 'UnloadExtAlg' );
           dummy = pd_call( 'LoadExtAlg', lib_name );
        end
      end
    end

   % Set initial values of the control algorithm
   dummy = pd_call( 'CallExtIPC', 'SetParam', ...
              [ 7 0 0 10 2.8 3 0.8 0.8 0.8 0.01 pi 0 ] );
   dummy = pd_call( 'SetSampleTime', 0.01 );
```

```
   dummy = pd_call( 'SetAlgNo', 99 );
   dummy = pd_call( 'ResetTime', 0 );

   % Wait for the first sample which may be send to the output
   while ( eq( pd_call( 'GetNoOfSamples', 0 ), 0 ) )
           ;
   end;
   history=pd_call( 'GetHistory' )';

   % initialize the initial conditions
   x0 = history( 1, 1 );

   str = [];    % str is always an empty matrix
   % initialize the array of sample times
   ts  = [-2 0];      % variable sample time


  case 2,     % Update - set new reference position
              % Used to set new reference position of the cart
     pars = pd_call( 'CallExtIPC', 'GetParam', [1] );
     pars( 12 ) = u;
     dummy = pd_call( 'CallExtIPC', 'SetParam', pars );
     sys = x;


  case 3,     % Outputs - return last samples
              % Used to transfer data to the output of the
              % Real Time Task block

    % Wait for the next sample transfered from the Real Time Task
to the
     % output of the S-function

    curr_len = size( history, 1 );
    if( curr_len < downsamp )
```

```
        while ( pd_call( 'GetNoOfSamples' ) + curr_len <= downsamp
)
            ;
        end;
       history = [ history; pd_call( 'GetHistory' )' ];
    end
     sys = history( downsamp, 2:7 );
    sys( 6 ) = u;
    curr_len = size( history, 1 );
    history = history( downsamp + 1 : curr_len, : );


  case 4,     % GetTimeOfNextVarHit %
    curr_len = size( history, 1 );
    if( curr_len < downsamp )
      while ( pd_call( 'GetNoOfSamples' ) + curr_len <= downsamp )
            ;
        end;
       history = [ history; pd_call( 'GetHistory' )' ];
    end
     sys = history( downsamp, 1 );


  case 9,     % Terminate
     dummy = pd_call( 'SetAlgNo', 0 );
     dummy = pd_call( 'UnloadExtAlg' );
    sys = [];

  case 1,     % Unhandled flags %
    sys = [];

  otherwise  % Unexpected flags %
    error(['Unexpected flag = ',num2str(flag)]);
end
```

10.2. INVERTED PENDULUM MODE SIMULINK MODEL

The Simulink model for the inverted pendulum mode is similar to the Simulink model for the crane mode and is shown in Fig. 10.7.

Notice, that the controller cannot swing up the pendulum from a down position to the upright position. The experiment must start with the pendulum placed in the neighbourhood of the upright equilibrium point. If the pendulum falls down during the experiment, it has to be turned up manually.

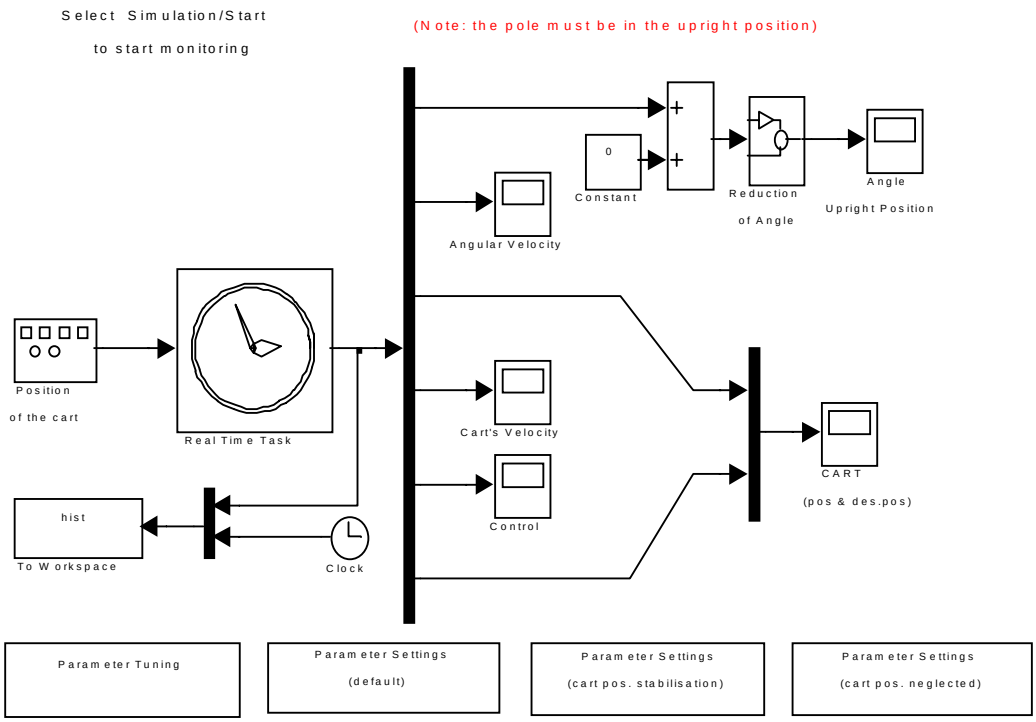


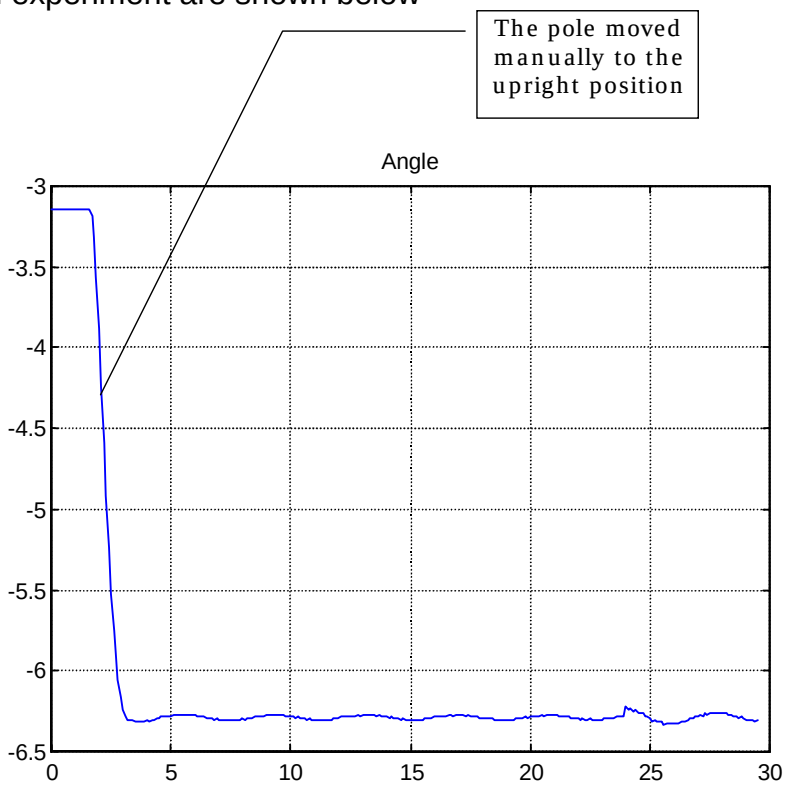
Fig. 10.7. Simulink model for the inverted pendulum mode (*pd_2dup.m* file).

The main differences between the controller for the crane mode and the controller for the inverted pendulum mode are the sets of parameters. The parameters for the inverted pendulum mode available in Simulink model (blocks *Parameter settings (default)*, *Parameter settings (cart pos. stabilisation)* and *Parameter settings (cart pos. neglected)* are shown in Table 10.2.

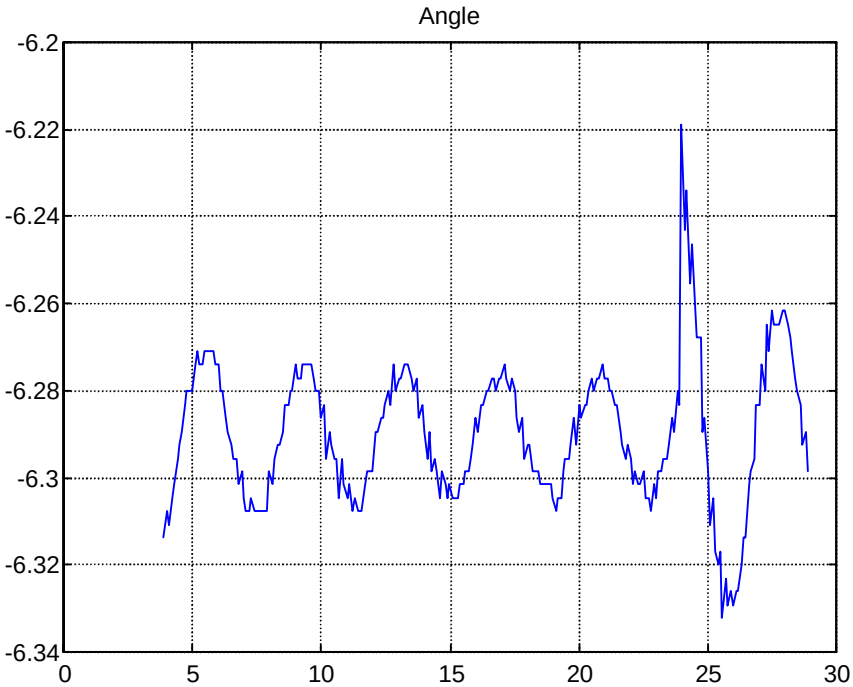
Table 10.2.

	Parameter Settings (default)	*Parameter Settings (cart. pos. stabilisation)*	*Parameter Settings (cart pos. neglected)*
KpAngle	-40.0	-8.0	-8.0
KiAngle	0.0	0.0	0.0
KdAngle	-1.0	-0.6	-0.6
KpPosition	-10.0	-3.0	-0.8
KiPosition	0.0	0.0	0.0
KdPosition	-2.0	-1.4	-1.4
IMaxAngle	0.8	0.8	0.8
IMaxPosition	0.8	0.8	0.8
UMax	0.8	0.8	0.8
TSample	0.01	0.01	0.01
RefAngle	0.0	0.0	0.0
RefPosition	0.0	0.0	0.0

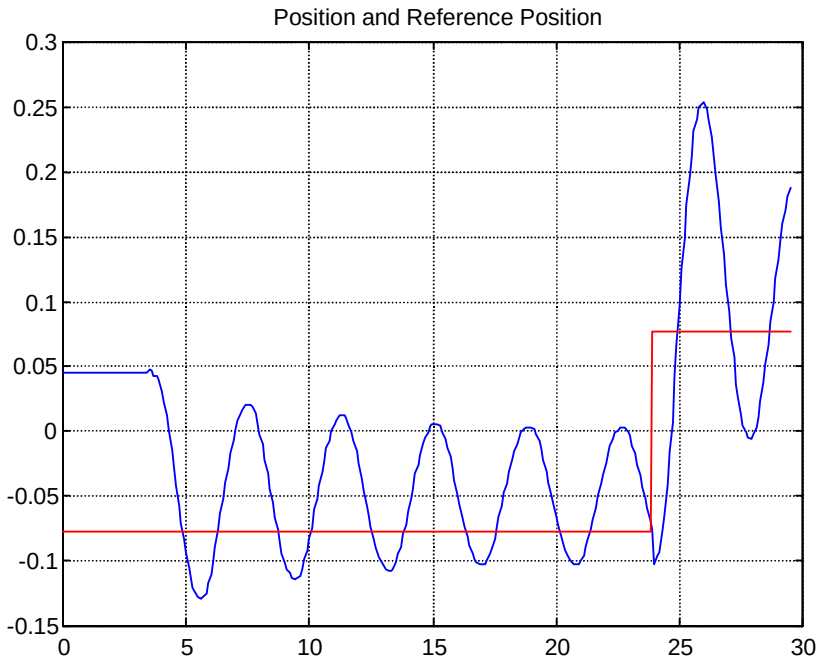
The results of an experiment are shown below



.plot(hist(:, 7), hist(:, 1)); grid; title('Angle')

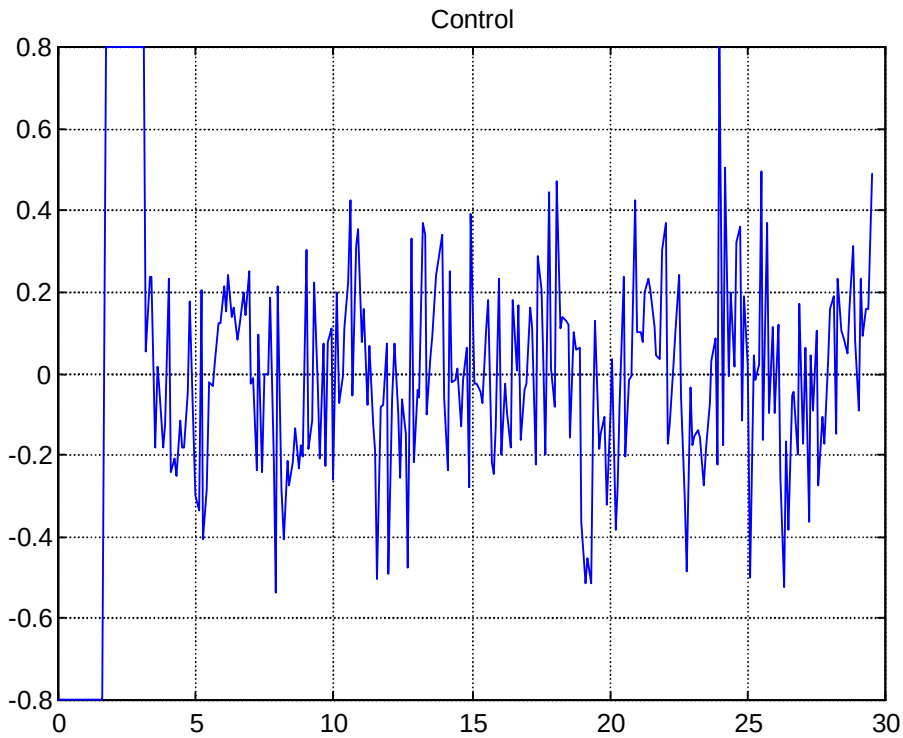


plot(hist(40:290, 7), hist(40:290, 1)); grid; title('Angle')



plot(hist(:, 7), hist(:, 3), hist(:, 7), hist(:, 6)); grid;

title('Position and Reference Position')



plot(hist(:, 7), hist(:, 5)); grid; title('Control')